

CFRM 421/521

[Eunice Seung-hyun Han]

Homework 4

- **Due: Tuesday, May 27, 2025, 11:59 PM**
- Total marks: 43
- Late submissions are allowed, but a 20% penalty per day applies. Your last submission is considered for calculating the penalty.
- Use this Jupyter notebook as a template for your solutions. **Your solution must be submitted as both one Jupyter notebook and one PDF file on Gradescope.** There will be two modules on Gradescope, one for each file type. The notebook must be already run, that is, make sure that you have run all the code, save the notebook, and then when you reopen the notebook, checked that all output appears as expected. You are allowed to use code from the textbook, textbook website, or lecture notes.

1. A regression MLP [12 marks]

Consider the original source of the California housing data (used in Homework 2) in Scikit-Learn. The data is obtained and split using the code below, where we split off 20% as the test set, and then split off 20% of the training set as a validation set, and keep the remaining 80% of the training set as the actual training set. The following code creates the training set `X_train`, `y_train`, the validation set `X_valid`, `y_valid` and the test set `X_test`, `y_test`.

```
In [ ]: import numpy as np
import pandas as pd
import tensorflow as tf
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split

housing = fetch_california_housing()
X = housing.data
```

```
y = housing.target

X_train_tmp, X_test, y_train_tmp, y_test = train_test_split(X, y, test
X_train, X_valid, y_train, y_valid = train_test_split(X_train_tmp, y_t
```

(a) [4 marks]

Use `tensorflow.keras` to train a regression MLP with a normalization layer as the first layer

(`tf.keras.layers.Normalization(input_shape=X_train.shape[1:])`), and one hidden layer of 50 ReLU neurons. For the output layer, try both a ReLU activation function and no activation function (which is equivalent to the identity function). Explain which choice is better. Use the appropriate weight initialization. Use the Nadam optimizer. Train for 30 epochs, and report the mean squared error on the validation set. In the `.compile()` method, use `loss="mse"`.

[Add your solution here]

```
In [ ]: # def fix_randomseeds(seed=42):
#         tf.random.set_seed(seed)
#         np.random.seed(seed)
#         tf.keras.backend.clear_session()
```

```
In [ ]: from tensorflow.keras import layers, models, initializers, optimizers

#fix_randomseeds()
model_relu_out = models.Sequential([tf.keras.layers.Normalization(input
                                tf.keras.layers.Dense(50, activati
                                tf.keras.layers.Dense(1, activatio
model_relu_out.compile(loss="mse", optimizer="nadam")
history_relu = model_relu_out.fit(X_train, y_train, validation_data=(X
mse_relu_out = model_relu_out.evaluate(X_valid, y_valid)

print(f"Validation MSE (ReLU output): {mse_relu_out}")
```

Epoch 1/30

/Users/eunicehan/.pyenv/versions/3.10.11/lib/python3.10/site-packages/k
eras/src/layers/preprocessing/tf_data_layer.py:19: UserWarning: Do not
pass an `input\_shape`/`input\_dim` argument to a layer. When using Seque
ntial models, prefer using an `Input(shape)` object as the first layer
in the model instead.

```
super().__init__(**kwargs)
```

413/413 ————— 1s 731us/step - loss: 5.7447 - val_loss:
5.7567

Epoch 2/30

413/413 ————— 0s 581us/step - loss: 5.5688 - val_loss:
5.7567

Epoch 3/30

413/413  **0s** 559us/step - loss: 5.5688 - val_loss: 5.7567
Epoch 4/30
413/413  **0s** 560us/step - loss: 5.5688 - val_loss: 5.7567
Epoch 5/30
413/413  **0s** 562us/step - loss: 5.5688 - val_loss: 5.7567
Epoch 6/30
413/413  **0s** 629us/step - loss: 5.5688 - val_loss: 5.7567
Epoch 7/30
413/413  **0s** 699us/step - loss: 5.5688 - val_loss: 5.7567
Epoch 8/30
413/413  **0s** 891us/step - loss: 5.5688 - val_loss: 5.7567
Epoch 9/30
413/413  **1s** 1ms/step - loss: 5.5688 - val_loss: 5.7567
Epoch 10/30
413/413  **0s** 603us/step - loss: 5.5688 - val_loss: 5.7567
Epoch 11/30
413/413  **0s** 635us/step - loss: 5.5688 - val_loss: 5.7567
Epoch 12/30
413/413  **0s** 576us/step - loss: 5.5688 - val_loss: 5.7567
Epoch 13/30
413/413  **0s** 590us/step - loss: 5.5688 - val_loss: 5.7567
Epoch 14/30
413/413  **0s** 579us/step - loss: 5.5688 - val_loss: 5.7567
Epoch 15/30
413/413  **0s** 568us/step - loss: 5.5688 - val_loss: 5.7567
Epoch 16/30
413/413  **0s** 566us/step - loss: 5.5688 - val_loss: 5.7567
Epoch 17/30
413/413  **0s** 572us/step - loss: 5.5688 - val_loss: 5.7567
Epoch 18/30
413/413  **0s** 569us/step - loss: 5.5688 - val_loss: 5.7567
Epoch 19/30
413/413  **0s** 627us/step - loss: 5.5688 - val_loss: 5.7567
Epoch 20/30
413/413  **0s** 571us/step - loss: 5.5688 - val_loss: 5.7567

```

5.7567
Epoch 21/30
413/413 ————— 0s 562us/step - loss: 5.5688 - val_loss:
5.7567
Epoch 22/30
413/413 ————— 0s 592us/step - loss: 5.5688 - val_loss:
5.7567
Epoch 23/30
413/413 ————— 0s 562us/step - loss: 5.5688 - val_loss:
5.7567
Epoch 24/30
413/413 ————— 0s 573us/step - loss: 5.5688 - val_loss:
5.7567
Epoch 25/30
413/413 ————— 0s 568us/step - loss: 5.5688 - val_loss:
5.7567
Epoch 26/30
413/413 ————— 0s 566us/step - loss: 5.5688 - val_loss:
5.7567
Epoch 27/30
413/413 ————— 0s 567us/step - loss: 5.5688 - val_loss:
5.7567
Epoch 28/30
413/413 ————— 0s 568us/step - loss: 5.5688 - val_loss:
5.7567
Epoch 29/30
413/413 ————— 0s 573us/step - loss: 5.5688 - val_loss:
5.7567
Epoch 30/30
413/413 ————— 0s 564us/step - loss: 5.5688 - val_loss:
5.7567
104/104 ————— 0s 678us/step - loss: 5.8433
Validation MSE (ReLU output): 5.756706237792969

```

```

In [ ]: #fix_randomseeds()
model_linear_out = models.Sequential([tf.keras.layers.Normalization(input_shape=(1,)),
                                     tf.keras.layers.Dense(50, activation='relu'),
                                     tf.keras.layers.Dense(1)])
model_linear_out.compile(loss="mse", optimizer="nadam")
history_linear = model_linear_out.fit(X_train, y_train, validation_data=(X_valid, y_valid))
mse_linear_out = model_linear_out.evaluate(X_valid, y_valid)

print(f"Validation MSE (linear output): {mse_linear_out}")

```

```

Epoch 1/30
413/413 ————— 1s 688us/step - loss: 18142.6680 - val_loss: 20.5527
Epoch 2/30
413/413 ————— 0s 551us/step - loss: 15.7674 - val_loss: 11.4502
Epoch 3/30
413/413 ————— 0s 576us/step - loss: 8.9536 - val_loss: 8.0570

```

Epoch 4/30
413/413  **0s** 553us/step - loss: 3.3094 - val_loss: 3.9322

Epoch 5/30
413/413  **0s** 551us/step - loss: 2.0957 - val_loss: 2.2288

Epoch 6/30
413/413  **0s** 556us/step - loss: 1.6586 - val_loss: 1.8055

Epoch 7/30
413/413  **0s** 555us/step - loss: 1.4157 - val_loss: 1.6261

Epoch 8/30
413/413  **0s** 551us/step - loss: 1.2799 - val_loss: 1.9917

Epoch 9/30
413/413  **0s** 553us/step - loss: 1.2233 - val_loss: 3.0113

Epoch 10/30
413/413  **0s** 557us/step - loss: 1.2203 - val_loss: 4.1373

Epoch 11/30
413/413  **0s** 578us/step - loss: 1.2281 - val_loss: 4.6236

Epoch 12/30
413/413  **0s** 557us/step - loss: 1.2299 - val_loss: 4.0238

Epoch 13/30
413/413  **0s** 553us/step - loss: 1.2582 - val_loss: 2.6011

Epoch 14/30
413/413  **0s** 557us/step - loss: 1.3663 - val_loss: 1.5367

Epoch 15/30
413/413  **0s** 556us/step - loss: 1.6163 - val_loss: 1.4861

Epoch 16/30
413/413  **0s** 557us/step - loss: 2.0723 - val_loss: 2.7408

Epoch 17/30
413/413  **0s** 555us/step - loss: 3.0829 - val_loss: 2.3984

Epoch 18/30
413/413  **0s** 553us/step - loss: 2.6024 - val_loss: 2.1051

Epoch 19/30
413/413  **0s** 556us/step - loss: 2.3538 - val_loss: 2.4376

Epoch 20/30
413/413  **0s** 558us/step - loss: 3.3102 - val_loss: 3.0661

Epoch 21/30

```

413/413 ————— 0s 557us/step - loss: 2.7294 - val_loss:
3.2115
Epoch 22/30
413/413 ————— 0s 619us/step - loss: 3.2687 - val_loss:
3.0737
Epoch 23/30
413/413 ————— 0s 555us/step - loss: 2.5986 - val_loss:
3.1166
Epoch 24/30
413/413 ————— 0s 555us/step - loss: 5.1109 - val_loss:
2.8091
Epoch 25/30
413/413 ————— 0s 564us/step - loss: 1.3397 - val_loss:
2.8106
Epoch 26/30
413/413 ————— 0s 556us/step - loss: 1.1319 - val_loss:
1.5339
Epoch 27/30
413/413 ————— 0s 557us/step - loss: 1.3657 - val_loss:
1.8836
Epoch 28/30
413/413 ————— 0s 561us/step - loss: 1.8714 - val_loss:
2.9791
Epoch 29/30
413/413 ————— 0s 562us/step - loss: 2.2310 - val_loss:
2.8903
Epoch 30/30
413/413 ————— 0s 564us/step - loss: 2.4173 - val_loss:
2.3627
104/104 ————— 0s 407us/step - loss: 2.0045
Validation MSE (linear output): 2.3627476692199707

```

Having the ReLU activation function on the output layer got the validation mean squared error: 5.756706237792969 and for not having an activation function on the output layer got the validation mean squared error: 2.3627476692199707.

This shows not having an activation function on the output layer is a better choice than having the ReLU activation function as it gives a smaller MSE.

(b) [6 marks]

Read the section "Fine-Tuning Neural Network Hyperparameters" in the textbook and the corresponding section in the [Jupyter notebook](#) on the textbook website using Keras Tuner. You will need to install the package `keras_tuner` if you don't already have it.

Then use Keras Tuner to do a randomized search to search for the best hyperparameters. Do the randomized search over the first 5000 observations of

the training set. Use 20 iterations, 20 epochs per iteration. Use the same network architecture as (a) except where otherwise specified below. Use no activation function for the output layer. Use a seed of 42, and the objective is clearly to minimize validation loss. The hyperparameters to search over are:

- Hidden layers: 1 to 5.
- Number of neurons per layer: 1 to 100.
- Learning rate: $1e-4$ to $1e-2$ using log sampling.
- ℓ_2 regularizers with `l2` value: $1e-4$ to 100 using log sampling.
- Optimizer:

```
tf.keras.optimizers.SGD(learning_rate=learning_rate,clipnorm=
and tf.keras.optimizers.Nadam(learning_rate=learning_rate) .
```

Print the best hyperparameter. (You can ignore any warning message you may get).

[Add your solution here]

```
In [ ]: X_train_red = X_train[:5000]
        y_train_red = y_train[:5000]
```

```
In [ ]: import keras_tuner as kt

# fix_randomseeds()

def build_model(hp):
    n_hidden = hp.Int("n_hidden", min_value=1, max_value=5)
    n_neurons = hp.Int("n_neurons", min_value=1, max_value=100)
    learning_rate = hp.Float("learning_rate", min_value=1e-4, max_valu
    l2_reg = hp.Float("l2", min_value=1e-4, max_value=100, sampling="l
    optimizer = hp.Choice("optimizer", values=["sgd", "nadam"])
    if optimizer == "sgd":
        optimizer = tf.keras.optimizers.SGD(learning_rate = learning_r
    else:
        optimizer = tf.keras.optimizers.Nadam(learning_rate = learning

    model = tf.keras.Sequential()
    model.add(tf.keras.layers.Normalization(input_shape=X_train.shape[

    for _ in range(n_hidden):
        model.add(tf.keras.layers.Dense(n_neurons, activation="relu",

    model.add(layers.Dense(1)) # output layer

    model.compile(loss="mse", optimizer=optimizer)

    return model
```

```
In [ ]: random_search_tuner = kt.RandomSearch(build_model, objective="val_loss",
random_search_tuner.search(X_train_red, y_train_red, epochs=20, validation_data=(X_val_red, y_val_red))
best_trial = random_search_tuner.oracle.get_best_trials(num_trials=1)[0]
print("")
best_trial.summary()
```

Trial 20 Complete [00h 00m 04s]

val_loss: 46.731136322021484

Best val_loss So Far: 0.6649941802024841

Total elapsed time: 00h 01m 17s

Trial 02 summary

Hyperparameters:

n_hidden: 5

n_neurons: 62

learning_rate: 0.006718710759425462

l2: 0.0003483686981793893

optimizer: nadam

Score: 0.6649941802024841

(c) [2 marks]

For the best model in (b), train the model on the full training data for 200 epochs.

Plot the learning curve. Does it look like the model is overfitting?

[Add your solution here]

```
In [ ]: # fix_randomseeds()
best_parameter = random_search_tuner.get_best_hyperparameters(num_trials=10)[0]

def get_best_model():
    model = tf.keras.Sequential()
    model.add(tf.keras.layers.Normalization(input_shape=X_train.shape[1:]))

    for _ in range(best_parameter.get('n_hidden')):
        model.add(tf.keras.layers.Dense(best_parameter.get('n_neurons')))
    model.add(layers.Dense(1))

    optimizer = best_parameter.get('optimizer')
    learning_rate = best_parameter.get('learning_rate')

    if optimizer == "sgd":
        optimizer = tf.keras.optimizers.SGD(learning_rate=learning_rate)
    else:
        optimizer = tf.keras.optimizers.Nadam(learning_rate=learning_rate)

    model.compile(loss="mse", optimizer=optimizer)
    return model
```

```
In [ ]: best_model = get_best_model()
        history = best_model.fit(X_train, y_train, epochs=200, validation_data
```

```
In [ ]: print(history.history)
```

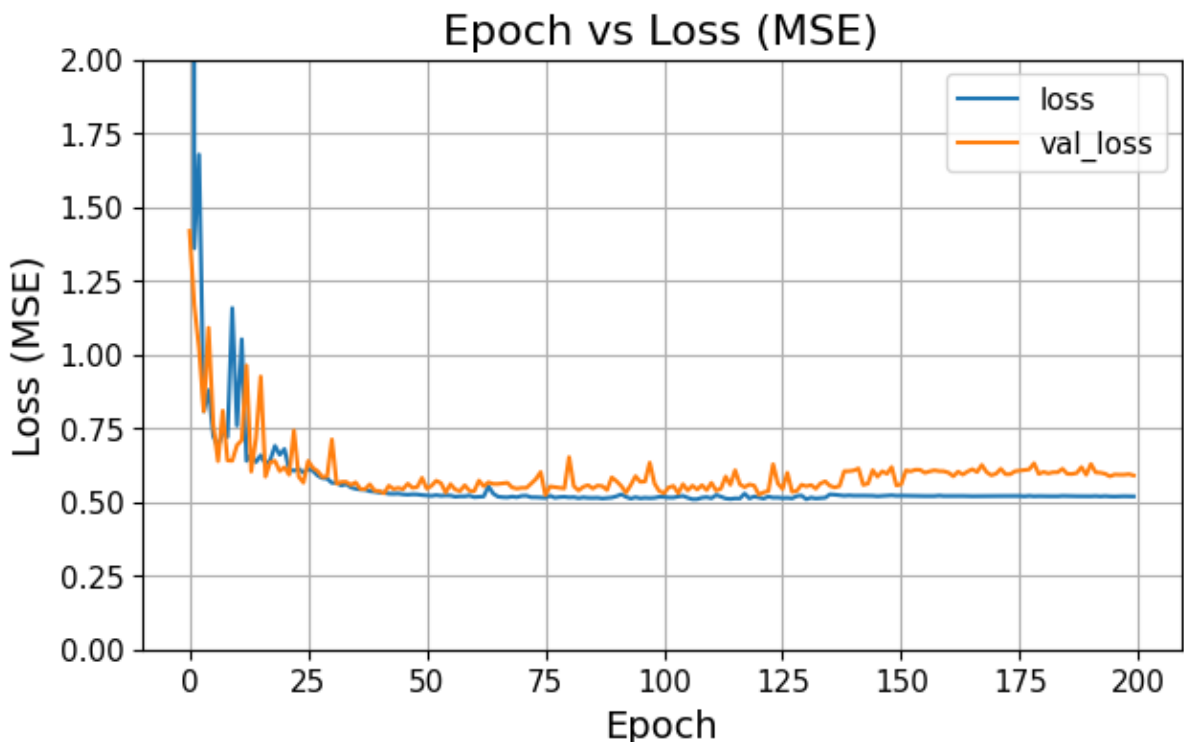
```
{'loss': [29.897260665893555, 1.359837532043457, 1.6778957843780518, 0.8077533841133118, 0.8789118528366089, 0.7228544354438782, 0.6867246627807617, 0.7388655543327332, 0.7205450534820557, 1.1575958728790283, 0.75913006067276, 1.0517466068267822, 0.6394625902175903, 0.6610135436058044, 0.6354718208312988, 0.6580919027328491, 0.6288736462593079, 0.6416705250740051, 0.6902749538421631, 0.660144567489624, 0.6796303987503052, 0.6096869707107544, 0.6058639287948608, 0.6118988990783691, 0.6000315546989441, 0.6122635006904602, 0.6056129932403564, 0.5893264412879944, 0.5808008909225464, 0.5795279145240784, 0.5647867321968079, 0.5656315088272095, 0.5557817220687866, 0.5608288049697876, 0.5496569871902466, 0.5448998212814331, 0.542296290397644, 0.5415163040161133, 0.536544680595398, 0.5350673198699951, 0.5323028564453125, 0.5314338803291321, 0.5281497240066528, 0.5276412963867188, 0.528432309627533, 0.5260356664657593, 0.5248063802719116, 0.5261485576629639, 0.5257272124290466, 0.5234038829803467, 0.5224245190620422, 0.5208261013031006, 0.5232659578323364, 0.5209905505180359, 0.5221657752990723, 0.5217474102973938, 0.5177738666534424, 0.5193818807601929, 0.5191462635993958, 0.521862804889679, 0.5165017247200012, 0.5177527666091919, 0.5187430381774902, 0.5531403422355652, 0.5285921096801758, 0.5181326866149902, 0.516355574131012, 0.5156276822090149, 0.5183629393577576, 0.5161653161048889, 0.52061527967453, 0.5212762355804443, 0.5159622430801392, 0.5159817934036255, 0.5147414207458496, 0.5141928791999817, 0.5212442278862, 0.5143455266952515, 0.5166700482368469, 0.517602801322937, 0.5155429840087891, 0.5165075063705444, 0.5130764245986938, 0.5156992077827454, 0.513988733291626, 0.5137618780136108, 0.5142996907234192, 0.5120685696601868, 0.5135918855667114, 0.5150296092033386, 0.5194190740585327, 0.525967538356781, 0.5172999501228333, 0.5115712285041809, 0.5161517858505249, 0.5124710202217102, 0.5142591595649719, 0.5132033228874207, 0.5141326785087585, 0.517417848110199, 0.5170321464538574, 0.515006422996521, 0.5149909853935242, 0.5183750987052917, 0.5206906199455261, 0.5143876671791077, 0.5093392729759216, 0.5095728635787964, 0.5146422386169434, 0.5165038704872131, 0.5125495195388794, 0.5235208868980408, 0.5181573629379272, 0.5119416117668152, 0.5102151036262512, 0.5123233795166016, 0.5118473768234253, 0.5285918116569519, 0.5120099782943726, 0.518596351146698, 0.5148172378540039, 0.511756956577301, 0.5182662606239319, 0.514768123626709, 0.5150384902954102, 0.5133711099624634, 0.5140463709831238, 0.5123432278633118, 0.5198293328285217, 0.5217992067337036, 0.5093429088592529, 0.5150059461593628, 0.5120501518249512, 0.5136386156082153, 0.5145583748817444, 0.525982141494751, 0.5247260332107544, 0.5227230787277222, 0.521148145198822, 0.5224665403366089, 0.5213761925697327, 0.5216604471206665, 0.5214643478393555, 0.5213422775268555, 0.5210195183753967, 0.5195727944374084, 0.5202401876449585, 0.5215536952018738, 0.5227123498916626, 0.5210056304931641, 0.5214283466339111, 0.5211231112480164, 0.5211570858955383, 0.5211388468742371, 0.520843505859375, 0.520060658454895, 0.5203248262405396, 0.520054042339325, 0.5216612815856934, 0.5202063322067261, 0.5202733874320984, 0.5204893946647644, 0.5201048254966736, 0.5203214883804321, 0.5196969509124756, 0.5196042060852051, 0.5198104381561279, 0.520604491
```

2338257, 0.5206054449081421, 0.5206908583641052, 0.5199545621871948, 0.5202712416648865, 0.5201967358589172, 0.5196649432182312, 0.5194732546806335, 0.5203836560249329, 0.519160807132721, 0.5208423733711243, 0.5193844437599182, 0.5199723839759827, 0.5194987654685974, 0.5194851160049438, 0.5193032622337341, 0.5194188356399536, 0.520611047744751, 0.5200719237327576, 0.5197692513465881, 0.5196225047111511, 0.5193605422973633, 0.5197620987892151, 0.5193935036659241, 0.5201336741447449, 0.5189375877380371, 0.5200251936912537, 0.5189028382301331, 0.5185659527778625, 0.5192283987998962, 0.5195032954216003, 0.5194234848022461, 0.5189182162284851], 'val_loss': [1.419142246246338, 1.1677426099777222, 1.0206544399261475, 0.8071867227554321, 1.0907015800476074, 0.7625792026519775, 0.6386340260505676, 0.8093416690826416, 0.6409298777580261, 0.6400008797645569, 0.6928306818008423, 0.7089856863021851, 0.9630321860313416, 0.603243887424469, 0.715233564376831, 0.9254043102264404, 0.585758626461029, 0.634750485420227, 0.6390690207481384, 0.6044808626174927, 0.6179655194282532, 0.5923506617546082, 0.7421365976333618, 0.5858985781669617, 0.5656472444534302, 0.6393688917160034, 0.6152836680412292, 0.6044020056724548, 0.584547221660614, 0.5868204236030579, 0.7120110988616943, 0.561860978603363, 0.5679022073745728, 0.5682083964347839, 0.5577477812767029, 0.5663415789604187, 0.5441346168518066, 0.5434447526931763, 0.5598003268241882, 0.538365364074707, 0.5371296405792236, 0.5296931862831116, 0.5553625226020813, 0.5439066886901855, 0.547706663608551, 0.5435411334037781, 0.5620917677879333, 0.5479834079742432, 0.5508213639259338, 0.5817455649375916, 0.5453200936317444, 0.5550685524940491, 0.5719268918037415, 0.5637696981430054, 0.5321621298789978, 0.5603564381599426, 0.5422918200492859, 0.5372166037559509, 0.5567541718482971, 0.5380728840827942, 0.5368929505348206, 0.5725162625312805, 0.5532599687576294, 0.5667033195495605, 0.5613083243370056, 0.5612969398498535, 0.5629663467407227, 0.5651212334632874, 0.5493764877319336, 0.5454670786857605, 0.5479007959365845, 0.5478044152259827, 0.5622001886367798, 0.5776399374008179, 0.6014531850814819, 0.5235776901245117, 0.5527386665344238, 0.5501709580421448, 0.5480622053146362, 0.546072781085968, 0.6521124243736267, 0.5536363124847412, 0.5392895936965942, 0.5514935851097107, 0.5569421052932739, 0.5458754301071167, 0.5556551814079285, 0.553642749786377, 0.5409908890724182, 0.5832640528678894, 0.5703927874565125, 0.5541974902153015, 0.5318478941917419, 0.557587742805481, 0.587491512298584, 0.5684328079223633, 0.5695734620094299, 0.6334944367408752, 0.5626369118690491, 0.5369858741760254, 0.5299257636070251, 0.5495291948318481, 0.5542328357696533, 0.529822051525116, 0.5603045225143433, 0.53927081823349, 0.5553857088088989, 0.5425163507461548, 0.5569584965705872, 0.5402950048446655, 0.5649911761283875, 0.5384986400604248, 0.5455598831176758, 0.5861373543739319, 0.558293879032135, 0.6077016592025757, 0.5597442388534546, 0.5502735376358032, 0.5594119429588318, 0.5557942390441895, 0.5250899195671082, 0.5324638485908508, 0.5371612310409546, 0.6282505393028259, 0.563008189201355, 0.5445219278335571, 0.5989998579025269, 0.5343246459960938, 0.5377165675163269, 0.5584809184074402, 0.5532665252685547, 0.5575570464134216, 0.5453647971153259, 0.5630210041999817, 0.5697159767150879, 0.55472332239151, 0.5494164824485779, 0.5540065169334412, 0.6043180823326111, 0.6045892238616943, 0.6067962050437927, 0.613678514957428, 0.558108925819397, 0.5624791979789734, 0.6080008149147034, 0.5854951739311218, 0.6040056347846985, 0.5956628918647766, 0.6183880567550659, 0.5562518835067749, 0.5600978136062622, 0.6071388721466064, 0.603984713

```
5543823, 0.6093215346336365, 0.6088627576828003, 0.5989550948143005, 0.6038385629653931, 0.6074619889259338, 0.604708731174469, 0.6041582226753235, 0.5992146134376526, 0.5872744917869568, 0.6028605103492737, 0.5930706262588501, 0.6032528281211853, 0.610762357711792, 0.6015968918800354, 0.6254852414131165, 0.6013712286949158, 0.5904296040534973, 0.5959432721138, 0.6123889684677124, 0.5941147208213806, 0.594227135181427, 0.6043427586555481, 0.606221616268158, 0.60748690366745, 0.6105079054832458, 0.6311287879943848, 0.59505295753479, 0.6018524765968323, 0.5969825387001038, 0.6006641387939453, 0.6008036136627197, 0.6126081347465515, 0.5954378843307495, 0.5949555039405823, 0.5981345176696777, 0.6180455684661865, 0.5922684669494629, 0.6287073493003845, 0.6003444790840149, 0.5992077589035034, 0.5954951047897339, 0.5869316458702087, 0.5930753946304321, 0.5923603177070618, 0.5924190282821655, 0.5943031907081604, 0.5898858308792114]}}
```

```
In [ ]: import matplotlib.pyplot as plt

pd.DataFrame(history.history).plot(figsize=(7,4), ylim=[0, 2], grid=True)
plt.xlabel("Epoch", fontsize=14)
plt.ylabel("Loss (MSE)", fontsize=14)
plt.title("Epoch vs Loss (MSE)", fontsize=16)
plt.legend(fontsize=11)
plt.show()
```



It seems like the model is not overfitting as the training and validation losses are converging and as epoch increases, they remain close to each other. Even the validation loss is slightly higher than the training loss, it is unclear to determine it is overfitting.

2. Binary classification DNN [17 marks]

Consider the [Portuguese Bank Marketing Data Set](#) available at Kaggle. Download the `bank_cleaned.csv` file or from [Canvas](#). Here we want to predict the success or failure of a bank marketing campaign using phone calls to promote a term deposit product. The target variable is `response_binary`.

The following code preprocesses the data. The day and month have been converted into cyclical features(1st day of the month has equal distance to the 2nd and the 31st).

```
In [ ]: df = pd.read_csv("bank_cleaned.csv")

month_dict = {"jan": 1, "feb": 2, "mar": 3, "apr": 4, "may": 5, "jun": 6,
              "jul": 7, "aug": 8, "sep": 9, "oct": 10, "nov": 11, "dec": 12}
day_rad = (df["day"] - 1) * (2 * np.pi / 31)
month_rad = (df["month"].replace(month_dict) - 1) * (2 * np.pi / 12)
df["day_sin"] = np.sin(day_rad)
df["day_cos"] = np.cos(day_rad)
df["month_sin"] = np.sin(month_rad)
df["month_cos"] = np.cos(month_rad)
df.drop(columns=["Unnamed: 0", "month", "day", "response"], axis=1, inplace=True)
df.head()
```

```
/var/folders/lc/16gprrls4_zff4bwgxvw2dnc0000gn/T/ipykernel_57393/648047190.py:6: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer_objects(copy=False)`. To opt-in to the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`
  month_rad = (df["month"].replace(month_dict) - 1) * (2 * np.pi / 12)
```

```
Out [ ]:   age   job   marital   education   default   balance   housing   loan   duration
```

	age	job	marital	education	default	balance	housing	loan	duration
0	58	management	married	tertiary	no	2143	yes	no	4
1	44	technician	single	secondary	no	29	yes	no	2
2	33	entrepreneur	married	secondary	no	2	yes	yes	1
3	35	management	married	tertiary	no	231	yes	no	2
4	28	management	single	tertiary	no	447	yes	yes	3

```
In [ ]: from sklearn.preprocessing import StandardScaler, OrdinalEncoder, OneHotEncoder
from sklearn.compose import ColumnTransformer

train_set_tmp, test_set = train_test_split(df, test_size=0.2, random_state=42)
train_set, valid_set = train_test_split(train_set_tmp, test_size=0.2,
```

```

X_train_raw = train_set.drop("response_binary", axis=1).copy()
y_train = train_set["response_binary"].copy()
X_valid_raw = valid_set.drop("response_binary", axis=1).copy()
y_valid = valid_set["response_binary"].copy()
X_test_raw = test_set.drop("response_binary", axis=1).copy()
y_test = test_set["response_binary"].copy()

num_attribs = list(X_train_raw.get_numeric_data().columns)
cat_attribs = list(set(X_train_raw.columns) - set(num_attribs))

cat_attribs_ord = ['default', 'housing', 'loan']
cat_attribs_hot = ['job', 'marital', 'education', 'poutcome']

full_pipeline = ColumnTransformer([
    ("num", StandardScaler(), num_attribs),
    ("cat_hot", OneHotEncoder(), cat_attribs_hot),
    ("cat_ord", OrdinalEncoder(categories=[['no', 'yes'], ['no', 'yes']]),
])

X_train = full_pipeline.fit_transform(X_train_raw)
X_valid = full_pipeline.transform(X_valid_raw)
X_test = full_pipeline.transform(X_test_raw)

```

(a) [4 marks]

In the next part you will build and fit a DNN with 4 hidden layers of 100 neurons each. Use the following specifications:

- (i) He initialization and the Swish activation function.
- (ii) The output layer has 1 neuron with sigmoid activation.
- (iii) Compile with `loss="binary_crossentropy"` and `metrics=["AUC"]`.

Explain why the choices (i), (ii), and (iii) are justified.

Also, state the proportion of successes in the training data.

[Add your solution here]

```
In [ ]: y_train.mean()
```

```
Out[ ]: np.float64(0.11168075907717029)
```

```

In [ ]: from tensorflow import keras

model = keras.Sequential([
    # (i)
    layers.Dense(100, activation=tf.nn.swish, kernel_initializer="he_n

```

```

layers.Dense(100, activation=tf.nn.swish, kernel_initializer="he_n
layers.Dense(100, activation=tf.nn.swish, kernel_initializer="he_n
layers.Dense(100, activation=tf.nn.swish, kernel_initializer="he_n
# output layer (ii)
layers.Dense(1, activation="sigmoid"))])

# (iii)
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=['
history = model.fit(X_train, y_train, validation_data=(X_valid, y_vali
success_rate = np.mean(y_train)
print(f"The proportion of suceses in the training data: {success_rate

```

Epoch 1/30

/Users/eunicehan/.pyenv/versions/3.10.11/lib/python3.10/site-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```

super().__init__(activity_regularizer=activity_regularizer, **kwargs)
817/817 ————— 1s 989us/step - AUC: 0.8542 - loss: 0.2589
- val_AUC: 0.9023 - val_loss: 0.2231

```

Epoch 2/30

```

817/817 ————— 1s 840us/step - AUC: 0.9118 - loss: 0.2144
- val_AUC: 0.9085 - val_loss: 0.2165

```

Epoch 3/30

```

817/817 ————— 1s 841us/step - AUC: 0.9195 - loss: 0.2066
- val_AUC: 0.9116 - val_loss: 0.2133

```

Epoch 4/30

```

817/817 ————— 1s 841us/step - AUC: 0.9246 - loss: 0.2010
- val_AUC: 0.9130 - val_loss: 0.2115

```

Epoch 5/30

```

817/817 ————— 1s 901us/step - AUC: 0.9292 - loss: 0.1956
- val_AUC: 0.9125 - val_loss: 0.2112

```

Epoch 6/30

```

817/817 ————— 1s 835us/step - AUC: 0.9341 - loss: 0.1900
- val_AUC: 0.9115 - val_loss: 0.2124

```

Epoch 7/30

```

817/817 ————— 1s 834us/step - AUC: 0.9382 - loss: 0.1842
- val_AUC: 0.9076 - val_loss: 0.2158

```

Epoch 8/30

```

817/817 ————— 1s 837us/step - AUC: 0.9428 - loss: 0.1781
- val_AUC: 0.9033 - val_loss: 0.2213

```

Epoch 9/30

```

817/817 ————— 1s 836us/step - AUC: 0.9470 - loss: 0.1715
- val_AUC: 0.8993 - val_loss: 0.2288

```

Epoch 10/30

```

817/817 ————— 1s 853us/step - AUC: 0.9517 - loss: 0.1642
- val_AUC: 0.8931 - val_loss: 0.2392

```

Epoch 11/30

```

817/817 ————— 1s 837us/step - AUC: 0.9563 - loss: 0.1564
- val_AUC: 0.8853 - val_loss: 0.2536

```

Epoch 12/30
817/817  **1s** 838us/step - AUC: 0.9610 - loss: 0.1485
- val_AUC: 0.8754 - val_loss: 0.2712

Epoch 13/30
817/817  **1s** 865us/step - AUC: 0.9655 - loss: 0.1402
- val_AUC: 0.8715 - val_loss: 0.2909

Epoch 14/30
817/817  **1s** 904us/step - AUC: 0.9702 - loss: 0.1308
- val_AUC: 0.8635 - val_loss: 0.3241

Epoch 15/30
817/817  **1s** 868us/step - AUC: 0.9742 - loss: 0.1229
- val_AUC: 0.8544 - val_loss: 0.3533

Epoch 16/30
817/817  **1s** 899us/step - AUC: 0.9772 - loss: 0.1139
- val_AUC: 0.8452 - val_loss: 0.3796

Epoch 17/30
817/817  **1s** 834us/step - AUC: 0.9804 - loss: 0.1061
- val_AUC: 0.8461 - val_loss: 0.3991

Epoch 18/30
817/817  **1s** 833us/step - AUC: 0.9834 - loss: 0.0975
- val_AUC: 0.8434 - val_loss: 0.4343

Epoch 19/30
817/817  **1s** 833us/step - AUC: 0.9865 - loss: 0.0887
- val_AUC: 0.8275 - val_loss: 0.4646

Epoch 20/30
817/817  **1s** 896us/step - AUC: 0.9890 - loss: 0.0816
- val_AUC: 0.8261 - val_loss: 0.5050

Epoch 21/30
817/817  **1s** 868us/step - AUC: 0.9896 - loss: 0.0786
- val_AUC: 0.8271 - val_loss: 0.4932

Epoch 22/30
817/817  **1s** 838us/step - AUC: 0.9920 - loss: 0.0701
- val_AUC: 0.8020 - val_loss: 0.5527

Epoch 23/30
817/817  **1s** 837us/step - AUC: 0.9929 - loss: 0.0637
- val_AUC: 0.8136 - val_loss: 0.5567

Epoch 24/30
817/817  **1s** 833us/step - AUC: 0.9931 - loss: 0.0631
- val_AUC: 0.8087 - val_loss: 0.5830

Epoch 25/30
817/817  **1s** 899us/step - AUC: 0.9933 - loss: 0.0595
- val_AUC: 0.7923 - val_loss: 0.6174

Epoch 26/30
817/817  **1s** 858us/step - AUC: 0.9943 - loss: 0.0548
- val_AUC: 0.7981 - val_loss: 0.6305

Epoch 27/30
817/817  **1s** 830us/step - AUC: 0.9949 - loss: 0.0531
- val_AUC: 0.7884 - val_loss: 0.6632

Epoch 28/30
817/817  **1s** 855us/step - AUC: 0.9960 - loss: 0.0467
- val_AUC: 0.7923 - val_loss: 0.6873

Epoch 29/30

```

817/817 _____ 1s 883us/step - AUC: 0.9960 - loss: 0.0439
- val_AUC: 0.7780 - val_loss: 0.7406
Epoch 30/30
817/817 _____ 1s 867us/step - AUC: 0.9966 - loss: 0.0397
- val_AUC: 0.7942 - val_loss: 0.7301
The proportion of successes in the training data: 0.11168075907717029

```

The reason why choice "(i) He initialization and the Swish activation function" is justified is because Swish has better performance for complex tasks and as it is a variant of ReLU activation function, the He initialization would be the accurate corresponding choice.

The reason why choice "(ii) The output layer has 1 neuron with sigmoid activation" is justified is because as we are predicting the variable, "response_binary," which is either 0 or 1, the output would be in between 0 and 1. And the activation that would be most accurate choice for this binary task would be the sigmoid activation.

The reason why choice "(iii) Compile with 'loss='binary_crossentropy'" and 'metrics=["AUC"]'" is justified is because in this binary classification task using "binary_crossentropy" loss with AUC (Area Under the Curve) metric allows to evaluate this model well and demonstrates how successfully the model was able to distinguish and determine between the two binary classes.

(b) [3 marks]

Train the model in (a) for 30 epochs and use exponential scheduling using the function below (`lr0=0.01` , `s=20`) and the NAG optimizer with `momentum=0.9` . Use a learning curve to comment on whether it is overfitting.

At the start of fitting your model, run `reset_session()` given by the following code.

```

In [ ]: def reset_session(seed=42):
        tf.random.set_seed(seed)
        np.random.seed(seed)
        tf.keras.backend.clear_session()

        def exponential_decay(lr0, s):
            return lambda epoch: lr0 * 0.1**(epoch / s)

```

[Add your solution here]

```

In [ ]: from tensorflow.keras import callbacks

```

```

reset_session()

model = tf.keras.models.Sequential([
    layers.Dense(100, activation='swish', kernel_initializer='he_normal'),
    layers.Dense(100, activation='swish', kernel_initializer='he_normal'),
    layers.Dense(100, activation='swish', kernel_initializer='he_normal'),
    layers.Dense(100, activation='swish', kernel_initializer='he_normal'),
    layers.Dense(1, activation='sigmoid')])

opt = optimizers.SGD(learning_rate=0.01, momentum=0.9, nesterov=True)
model.compile(optimizer=opt, loss='binary_crossentropy', metrics=['AUC'])

lr_scheduler = callbacks.LearningRateScheduler(exponential_decay(lr0=0.01))

history_b = model.fit(X_train, y_train, validation_data=(X_valid, y_val),

```

Epoch 1/30

817/817 ————— **1s** 938us/step - AUC: 0.8045 - loss: 0.2933
 - val_AUC: 0.8939 - val_loss: 0.2285 - learning_rate: 0.0100

Epoch 2/30

817/817 ————— **1s** 773us/step - AUC: 0.9043 - loss: 0.2209
 - val_AUC: 0.9014 - val_loss: 0.2218 - learning_rate: 0.0089

Epoch 3/30

817/817 ————— **1s** 772us/step - AUC: 0.9107 - loss: 0.2149
 - val_AUC: 0.9044 - val_loss: 0.2184 - learning_rate: 0.0079

Epoch 4/30

817/817 ————— **1s** 798us/step - AUC: 0.9146 - loss: 0.2111
 - val_AUC: 0.9072 - val_loss: 0.2164 - learning_rate: 0.0071

Epoch 5/30

817/817 ————— **1s** 774us/step - AUC: 0.9176 - loss: 0.2081
 - val_AUC: 0.9084 - val_loss: 0.2150 - learning_rate: 0.0063

Epoch 6/30

817/817 ————— **1s** 770us/step - AUC: 0.9203 - loss: 0.2056
 - val_AUC: 0.9095 - val_loss: 0.2140 - learning_rate: 0.0056

Epoch 7/30

817/817 ————— **1s** 798us/step - AUC: 0.9222 - loss: 0.2033
 - val_AUC: 0.9103 - val_loss: 0.2132 - learning_rate: 0.0050

Epoch 8/30

817/817 ————— **1s** 770us/step - AUC: 0.9239 - loss: 0.2013
 - val_AUC: 0.9111 - val_loss: 0.2126 - learning_rate: 0.0045

Epoch 9/30

817/817 ————— **1s** 769us/step - AUC: 0.9255 - loss: 0.1995
 - val_AUC: 0.9109 - val_loss: 0.2122 - learning_rate: 0.0040

Epoch 10/30

817/817 ————— **1s** 770us/step - AUC: 0.9267 - loss: 0.1978
 - val_AUC: 0.9109 - val_loss: 0.2119 - learning_rate: 0.0035

Epoch 11/30

817/817 ————— **1s** 769us/step - AUC: 0.9279 - loss: 0.1963
 - val_AUC: 0.9113 - val_loss: 0.2117 - learning_rate: 0.0032

Epoch 12/30

817/817 ————— **1s** 773us/step - AUC: 0.9292 - loss: 0.1950
 - val_AUC: 0.9118 - val_loss: 0.2116 - learning_rate: 0.0028

Epoch 13/30

817/817 ————— **1s** 779us/step - AUC: 0.9302 - loss: 0.1937
- val_AUC: 0.9123 - val_loss: 0.2115 - learning_rate: 0.0025
Epoch 14/30

817/817 ————— **1s** 771us/step - AUC: 0.9312 - loss: 0.1926
- val_AUC: 0.9125 - val_loss: 0.2115 - learning_rate: 0.0022
Epoch 15/30

817/817 ————— **1s** 772us/step - AUC: 0.9320 - loss: 0.1915
- val_AUC: 0.9123 - val_loss: 0.2114 - learning_rate: 0.0020
Epoch 16/30

817/817 ————— **1s** 769us/step - AUC: 0.9328 - loss: 0.1905
- val_AUC: 0.9124 - val_loss: 0.2114 - learning_rate: 0.0018
Epoch 17/30

817/817 ————— **1s** 768us/step - AUC: 0.9335 - loss: 0.1897
- val_AUC: 0.9126 - val_loss: 0.2115 - learning_rate: 0.0016
Epoch 18/30

817/817 ————— **1s** 799us/step - AUC: 0.9341 - loss: 0.1888
- val_AUC: 0.9129 - val_loss: 0.2115 - learning_rate: 0.0014
Epoch 19/30

817/817 ————— **1s** 771us/step - AUC: 0.9347 - loss: 0.1881
- val_AUC: 0.9128 - val_loss: 0.2115 - learning_rate: 0.0013
Epoch 20/30

817/817 ————— **1s** 771us/step - AUC: 0.9353 - loss: 0.1874
- val_AUC: 0.9128 - val_loss: 0.2115 - learning_rate: 0.0011
Epoch 21/30

817/817 ————— **1s** 771us/step - AUC: 0.9358 - loss: 0.1868
- val_AUC: 0.9126 - val_loss: 0.2116 - learning_rate: 0.0010
Epoch 22/30

817/817 ————— **1s** 767us/step - AUC: 0.9363 - loss: 0.1862
- val_AUC: 0.9128 - val_loss: 0.2116 - learning_rate: 8.9125e-04
Epoch 23/30

817/817 ————— **1s** 798us/step - AUC: 0.9367 - loss: 0.1857
- val_AUC: 0.9129 - val_loss: 0.2117 - learning_rate: 7.9433e-04
Epoch 24/30

817/817 ————— **1s** 765us/step - AUC: 0.9371 - loss: 0.1852
- val_AUC: 0.9131 - val_loss: 0.2117 - learning_rate: 7.0795e-04
Epoch 25/30

817/817 ————— **1s** 770us/step - AUC: 0.9374 - loss: 0.1847
- val_AUC: 0.9131 - val_loss: 0.2118 - learning_rate: 6.3096e-04
Epoch 26/30

817/817 ————— **1s** 764us/step - AUC: 0.9377 - loss: 0.1843
- val_AUC: 0.9132 - val_loss: 0.2118 - learning_rate: 5.6234e-04
Epoch 27/30

817/817 ————— **1s** 785us/step - AUC: 0.9380 - loss: 0.1839
- val_AUC: 0.9133 - val_loss: 0.2119 - learning_rate: 5.0119e-04
Epoch 28/30

817/817 ————— **1s** 806us/step - AUC: 0.9383 - loss: 0.1836
- val_AUC: 0.9133 - val_loss: 0.2119 - learning_rate: 4.4668e-04
Epoch 29/30

817/817 ————— **1s** 767us/step - AUC: 0.9385 - loss: 0.1833
- val_AUC: 0.9134 - val_loss: 0.2120 - learning_rate: 3.9811e-04
Epoch 30/30

817/817 ————— **1s** 770us/step - AUC: 0.9384 - loss: 0.1830

- val_AUC: 0.9136 - val_loss: 0.2120 - learning_rate: 3.5481e-04

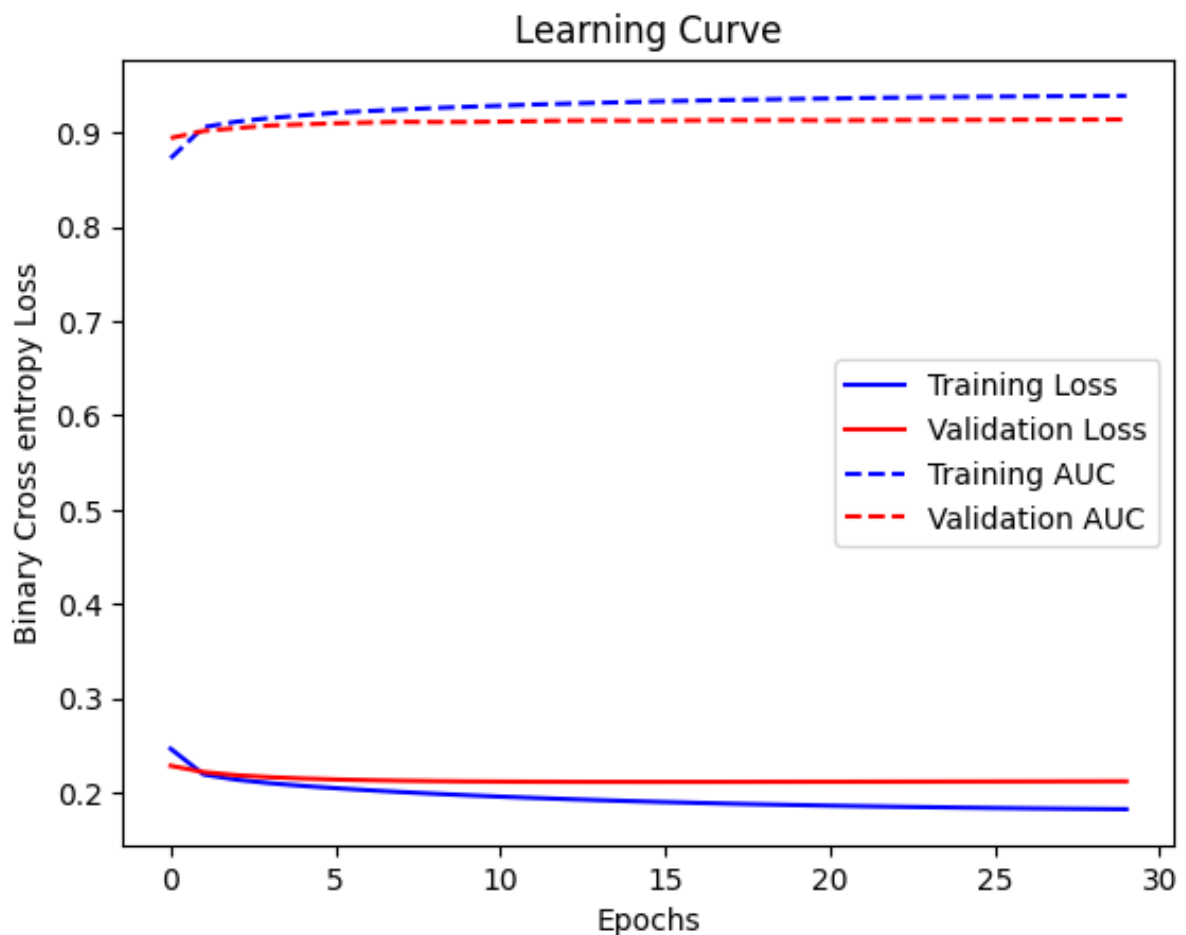
```
In [ ]: val_auc = model.evaluate(X_valid, y_valid)[1]
```

```
print("The validation AUC of the model (b): ", val_auc)
```

205/205 ————— 0s 507us/step - AUC: 0.9134 - loss: 0.2177
The validation AUC of the model (b): 0.9136109352111816

```
In [ ]: # Learning curve
```

```
plt.figure()
plt.plot(history_b.history["loss"], color='blue', label="Training Loss")
plt.plot(history_b.history["val_loss"], color='red', label="Validation Loss")
plt.plot(history_b.history["AUC"], linestyle='--', color='blue', label="Training AUC")
plt.plot(history_b.history["val_AUC"], linestyle='--', color='red', label="Validation AUC")
plt.xlabel("Epochs")
plt.ylabel("Binary Cross entropy Loss")
plt.legend()
plt.title("Learning Curve")
plt.show()
```



Looking on the learning curve, it seems like the validation loss is greater than the training loss when the number of epochs is increasing. And at the same time, the validation AUC is smaller than the training AUC when the number of epochs is

increasing. This widening gap between the curves show that it is overfitting as it is fitting the training data better than it generalizes to new data (validation data).

(c) [8 marks]

Fit separate models using the same specification as in (b) but with the following regularization techniques:

- (i) batch normalization,
- (ii) early stopping based on validation AUC with `patience=10` (look at the documentation and note the `mode` argument).
- (iii) ℓ_2 regularization with `l2=0.0002`,
- (iv) dropout with probability 0.02,
- (v) ℓ_2 regularization and early stopping both as above,
- (vi) batch normalization and dropout both as above.

At the start of each one of the above models, run `reset_session()`.

The performance measure is validation AUC. State this for the model in (b), and for each of the models here comment on whether it is better than the model in (b).

[Add your solution here]

The validation AUC of the model (b): 0.9136109352111816

```
In [ ]: # (i) batch normalization

reset_session()

model_bn = tf.keras.models.Sequential([
    layers.Dense(100, activation='swish', kernel_initializer='he_normal'),
    layers.BatchNormalization(),
    layers.Dense(100, activation='swish', kernel_initializer='he_normal'),
    layers.BatchNormalization(),
    layers.Dense(100, activation='swish', kernel_initializer='he_normal'),
    layers.BatchNormalization(),
    layers.Dense(100, activation='swish', kernel_initializer='he_normal'),
    layers.BatchNormalization(),
    layers.Dense(1, activation='sigmoid')
])
```

```

opt = optimizers.SGD(learning_rate=0.01, momentum=0.9, nesterov=True)
model_bn.compile(optimizer=opt, loss="binary_crossentropy", metrics=['

lr_scheduler = callbacks.LearningRateScheduler(exponential_decay(lr0=0

history_bn = model_bn.fit(X_train, y_train, validation_data=(X_valid,

val_auc_bn = model_bn.evaluate(X_valid, y_valid)[1]

```

Epoch 1/30

817/817 ————— 2s 1ms/step - AUC: 0.8181 - loss: 0.3136 - val_AUC: 0.8929 - val_loss: 0.2370 - learning_rate: 0.0100

Epoch 2/30

817/817 ————— 1s 964us/step - AUC: 0.9125 - loss: 0.2142 - val_AUC: 0.8967 - val_loss: 0.2386 - learning_rate: 0.0089

Epoch 3/30

817/817 ————— 1s 984us/step - AUC: 0.9274 - loss: 0.1976 - val_AUC: 0.8962 - val_loss: 0.2457 - learning_rate: 0.0079

Epoch 4/30

817/817 ————— 1s 964us/step - AUC: 0.9385 - loss: 0.1835 - val_AUC: 0.8926 - val_loss: 0.2569 - learning_rate: 0.0071

Epoch 5/30

817/817 ————— 1s 961us/step - AUC: 0.9487 - loss: 0.1690 - val_AUC: 0.8887 - val_loss: 0.2704 - learning_rate: 0.0063

Epoch 6/30

817/817 ————— 1s 959us/step - AUC: 0.9586 - loss: 0.1533 - val_AUC: 0.8835 - val_loss: 0.2847 - learning_rate: 0.0056

Epoch 7/30

817/817 ————— 1s 992us/step - AUC: 0.9675 - loss: 0.1367 - val_AUC: 0.8788 - val_loss: 0.2991 - learning_rate: 0.0050

Epoch 8/30

817/817 ————— 1s 961us/step - AUC: 0.9754 - loss: 0.1197 - val_AUC: 0.8714 - val_loss: 0.3144 - learning_rate: 0.0045

Epoch 9/30

817/817 ————— 1s 964us/step - AUC: 0.9826 - loss: 0.1024 - val_AUC: 0.8642 - val_loss: 0.3311 - learning_rate: 0.0040

Epoch 10/30

817/817 ————— 1s 1ms/step - AUC: 0.9881 - loss: 0.0860 - val_AUC: 0.8552 - val_loss: 0.3497 - learning_rate: 0.0035

Epoch 11/30

817/817 ————— 1s 1ms/step - AUC: 0.9926 - loss: 0.0711 - val_AUC: 0.8525 - val_loss: 0.3707 - learning_rate: 0.0032

Epoch 12/30

817/817 ————— 1s 1ms/step - AUC: 0.9955 - loss: 0.0581 - val_AUC: 0.8464 - val_loss: 0.3921 - learning_rate: 0.0028

Epoch 13/30

817/817 ————— 1s 1ms/step - AUC: 0.9975 - loss: 0.0471 - val_AUC: 0.8400 - val_loss: 0.4134 - learning_rate: 0.0025

Epoch 14/30

817/817 ————— 1s 1ms/step - AUC: 0.9986 - loss: 0.0380 - val_AUC: 0.8361 - val_loss: 0.4308 - learning_rate: 0.0022

Epoch 15/30

817/817 ————— 1s 1ms/step - AUC: 0.9992 - loss: 0.0309 -

```

val_AUC: 0.8333 - val_loss: 0.4448 - learning_rate: 0.0020
Epoch 16/30
817/817  1s 960us/step - AUC: 0.9995 - loss: 0.0255
- val_AUC: 0.8308 - val_loss: 0.4575 - learning_rate: 0.0018
Epoch 17/30
817/817  1s 959us/step - AUC: 0.9997 - loss: 0.0214
- val_AUC: 0.8305 - val_loss: 0.4691 - learning_rate: 0.0016
Epoch 18/30
817/817  1s 958us/step - AUC: 0.9998 - loss: 0.0184
- val_AUC: 0.8264 - val_loss: 0.4799 - learning_rate: 0.0014
Epoch 19/30
817/817  1s 986us/step - AUC: 0.9999 - loss: 0.0161
- val_AUC: 0.8239 - val_loss: 0.4895 - learning_rate: 0.0013
Epoch 20/30
817/817  1s 953us/step - AUC: 0.9999 - loss: 0.0144
- val_AUC: 0.8225 - val_loss: 0.4979 - learning_rate: 0.0011
Epoch 21/30
817/817  1s 955us/step - AUC: 0.9999 - loss: 0.0130
- val_AUC: 0.8201 - val_loss: 0.5052 - learning_rate: 0.0010
Epoch 22/30
817/817  1s 955us/step - AUC: 1.0000 - loss: 0.0119
- val_AUC: 0.8164 - val_loss: 0.5116 - learning_rate: 8.9125e-04
Epoch 23/30
817/817  1s 984us/step - AUC: 1.0000 - loss: 0.0110
- val_AUC: 0.8155 - val_loss: 0.5171 - learning_rate: 7.9433e-04
Epoch 24/30
817/817  1s 974us/step - AUC: 1.0000 - loss: 0.0103
- val_AUC: 0.8120 - val_loss: 0.5220 - learning_rate: 7.0795e-04
Epoch 25/30
817/817  1s 959us/step - AUC: 1.0000 - loss: 0.0097
- val_AUC: 0.8117 - val_loss: 0.5262 - learning_rate: 6.3096e-04
Epoch 26/30
817/817  1s 980us/step - AUC: 1.0000 - loss: 0.0092
- val_AUC: 0.8112 - val_loss: 0.5299 - learning_rate: 5.6234e-04
Epoch 27/30
817/817  1s 956us/step - AUC: 1.0000 - loss: 0.0088
- val_AUC: 0.8115 - val_loss: 0.5331 - learning_rate: 5.0119e-04
Epoch 28/30
817/817  1s 951us/step - AUC: 1.0000 - loss: 0.0084
- val_AUC: 0.8116 - val_loss: 0.5359 - learning_rate: 4.4668e-04
Epoch 29/30
817/817  1s 955us/step - AUC: 1.0000 - loss: 0.0081
- val_AUC: 0.8111 - val_loss: 0.5384 - learning_rate: 3.9811e-04
Epoch 30/30
817/817  1s 950us/step - AUC: 1.0000 - loss: 0.0078
- val_AUC: 0.8119 - val_loss: 0.5405 - learning_rate: 3.5481e-04
205/205  0s 518us/step - AUC: 0.8032 - loss: 0.5466

```

```
In [ ]: print("The validation AUC of the model with batch normalization: ", va
```

```
The validation AUC of the model with batch normalization: 0.8118569850
921631
```

The validation AUC of the model (b): 0.9136109352111816

The validation AUC of the model with batch normalization: 0.8118569850921631

Comparing the model in (b) with the model using batch normalization regularization technique, it seems like the validation AUC is higher for model (b), which tells the model with batch normalization is NOT better than the model in (b).

```
In [ ]: # (ii) early stopping based on validation AUC with `patience=10` (look
reset_session()

model_es = tf.keras.models.Sequential([
    layers.Dense(100, activation='swish', kernel_initializer="he_normal"),
    layers.Dense(100, activation='swish', kernel_initializer="he_normal"),
    layers.Dense(100, activation='swish', kernel_initializer="he_normal"),
    layers.Dense(100, activation='swish', kernel_initializer="he_normal"),
    layers.Dense(1, activation="sigmoid")
])

opt = optimizers.SGD(learning_rate=0.01, momentum=0.9, nesterov=True)
model_es.compile(optimizer=opt, loss="binary_crossentropy", metrics=['
lr_scheduler = callbacks.LearningRateScheduler(exponential_decay(lr0=0
early_stop = callbacks.EarlyStopping(monitor='val_auc', patience=10, m
history_es = model_es.fit(X_train, y_train, validation_data=(X_valid,
val_auc_es = model_es.evaluate(X_valid, y_valid)[1])
```

Epoch 1/30

817/817 ————— **1s** 902us/step - AUC: 0.8279 - loss: 0.2806
- val_AUC: 0.8944 - val_loss: 0.2293 - learning_rate: 0.0100

Epoch 2/30

245/817 ————— **0s** 619us/step - AUC: 0.9046 - loss: 0.2209

/Users/eunicehan/.pyenv/versions/3.10.11/lib/python3.10/site-packages/k
eras/src/callbacks/early_stopping.py:153: UserWarning: Early stopping c
onditioned on metric `val\_auc` which is not available. Available metric
s are: AUC, loss, val_AUC, val_loss, learning_rate
current = self.get_monitor_value(logs)

817/817 ————— **1s** 778us/step - AUC: 0.9057 - loss: 0.2197
- val_AUC: 0.9008 - val_loss: 0.2239 - learning_rate: 0.0089

Epoch 3/30

817/817 ————— **1s** 773us/step - AUC: 0.9118 - loss: 0.2140
- val_AUC: 0.9042 - val_loss: 0.2209 - learning_rate: 0.0079

Epoch 4/30

817/817 ————— **1s** 767us/step - AUC: 0.9159 - loss: 0.2101
- val_AUC: 0.9065 - val_loss: 0.2187 - learning_rate: 0.0071

Epoch 5/30

817/817 ————— **1s** 767us/step - AUC: 0.9187 - loss: 0.2069
- val_AUC: 0.9085 - val_loss: 0.2169 - learning_rate: 0.0063
Epoch 6/30

817/817 ————— **1s** 791us/step - AUC: 0.9213 - loss: 0.2042
- val_AUC: 0.9101 - val_loss: 0.2155 - learning_rate: 0.0056
Epoch 7/30

817/817 ————— **1s** 760us/step - AUC: 0.9236 - loss: 0.2019
- val_AUC: 0.9111 - val_loss: 0.2143 - learning_rate: 0.0050
Epoch 8/30

817/817 ————— **1s** 764us/step - AUC: 0.9255 - loss: 0.1998
- val_AUC: 0.9117 - val_loss: 0.2133 - learning_rate: 0.0045
Epoch 9/30

817/817 ————— **1s** 759us/step - AUC: 0.9272 - loss: 0.1979
- val_AUC: 0.9128 - val_loss: 0.2125 - learning_rate: 0.0040
Epoch 10/30

817/817 ————— **1s** 762us/step - AUC: 0.9285 - loss: 0.1963
- val_AUC: 0.9134 - val_loss: 0.2119 - learning_rate: 0.0035
Epoch 11/30

817/817 ————— **1s** 757us/step - AUC: 0.9299 - loss: 0.1948
- val_AUC: 0.9138 - val_loss: 0.2114 - learning_rate: 0.0032
Epoch 12/30

817/817 ————— **1s** 790us/step - AUC: 0.9311 - loss: 0.1935
- val_AUC: 0.9141 - val_loss: 0.2110 - learning_rate: 0.0028
Epoch 13/30

817/817 ————— **1s** 762us/step - AUC: 0.9321 - loss: 0.1922
- val_AUC: 0.9145 - val_loss: 0.2107 - learning_rate: 0.0025
Epoch 14/30

817/817 ————— **1s** 766us/step - AUC: 0.9329 - loss: 0.1911
- val_AUC: 0.9150 - val_loss: 0.2104 - learning_rate: 0.0022
Epoch 15/30

817/817 ————— **1s** 759us/step - AUC: 0.9337 - loss: 0.1901
- val_AUC: 0.9152 - val_loss: 0.2103 - learning_rate: 0.0020
Epoch 16/30

817/817 ————— **1s** 762us/step - AUC: 0.9345 - loss: 0.1892
- val_AUC: 0.9156 - val_loss: 0.2102 - learning_rate: 0.0018
Epoch 17/30

817/817 ————— **1s** 761us/step - AUC: 0.9352 - loss: 0.1883
- val_AUC: 0.9157 - val_loss: 0.2101 - learning_rate: 0.0016
Epoch 18/30

817/817 ————— **1s** 764us/step - AUC: 0.9357 - loss: 0.1875
- val_AUC: 0.9156 - val_loss: 0.2100 - learning_rate: 0.0014
Epoch 19/30

817/817 ————— **1s** 770us/step - AUC: 0.9364 - loss: 0.1868
- val_AUC: 0.9159 - val_loss: 0.2100 - learning_rate: 0.0013
Epoch 20/30

817/817 ————— **1s** 759us/step - AUC: 0.9369 - loss: 0.1861
- val_AUC: 0.9157 - val_loss: 0.2100 - learning_rate: 0.0011
Epoch 21/30

817/817 ————— **1s** 759us/step - AUC: 0.9374 - loss: 0.1855
- val_AUC: 0.9156 - val_loss: 0.2101 - learning_rate: 0.0010
Epoch 22/30

817/817 ————— **1s** 762us/step - AUC: 0.9378 - loss: 0.1849

```

- val_AUC: 0.9158 - val_loss: 0.2101 - learning_rate: 8.9125e-04
Epoch 23/30
817/817 ————— 1s 761us/step - AUC: 0.9382 - loss: 0.1844
- val_AUC: 0.9157 - val_loss: 0.2102 - learning_rate: 7.9433e-04
Epoch 24/30
817/817 ————— 1s 790us/step - AUC: 0.9385 - loss: 0.1839
- val_AUC: 0.9159 - val_loss: 0.2103 - learning_rate: 7.0795e-04
Epoch 25/30
817/817 ————— 1s 761us/step - AUC: 0.9389 - loss: 0.1835
- val_AUC: 0.9159 - val_loss: 0.2103 - learning_rate: 6.3096e-04
Epoch 26/30
817/817 ————— 1s 792us/step - AUC: 0.9392 - loss: 0.1831
- val_AUC: 0.9160 - val_loss: 0.2104 - learning_rate: 5.6234e-04
Epoch 27/30
817/817 ————— 1s 768us/step - AUC: 0.9394 - loss: 0.1828
- val_AUC: 0.9161 - val_loss: 0.2105 - learning_rate: 5.0119e-04
Epoch 28/30
817/817 ————— 1s 792us/step - AUC: 0.9397 - loss: 0.1824
- val_AUC: 0.9161 - val_loss: 0.2106 - learning_rate: 4.4668e-04
Epoch 29/30
817/817 ————— 1s 764us/step - AUC: 0.9399 - loss: 0.1821
- val_AUC: 0.9160 - val_loss: 0.2107 - learning_rate: 3.9811e-04
Epoch 30/30
817/817 ————— 1s 758us/step - AUC: 0.9401 - loss: 0.1819
- val_AUC: 0.9159 - val_loss: 0.2108 - learning_rate: 3.5481e-04
205/205 ————— 0s 493us/step - AUC: 0.9147 - loss: 0.2181

```

```
In [ ]: print("The validation AUC of the model with early stopping: ", val_auc
```

```
The validation AUC of the model with early stopping: 0.9159131050109863
```

The validation AUC of the model (b): 0.9136109352111816

The validation AUC of the model with early stopping: 0.9159131050109863

Comparing the model in (b) with the model using early stopping regularization technique, it seems like the validation AUC is slightly higher for the model using early stopping, which tells the model with early stopping is better than the model in (b).

```
In [ ]: # (iii) $ell_2$ regularization with `l2=0.0002`

reset_session()

l2_reg = tf.keras.regularizers.l2(0.0002)

model_l2 = tf.keras.models.Sequential([
    layers.Dense(100, activation='swish', kernel_initializer='he_normal',
    layers.Dense(100, activation='swish', kernel_initializer='he_normal',
    layers.Dense(100, activation='swish', kernel_initializer='he_normal',
    layers.Dense(100, activation='swish', kernel_initializer='he_normal',

```

```

layers.Dense(1, activation="sigmoid")
])

opt = optimizers.SGD(learning_rate=0.01, momentum=0.9, nesterov=True)
model_l2.compile(optimizer=opt, loss="binary_crossentropy", metrics=['

lr_scheduler = callbacks.LearningRateScheduler(exponential_decay(lr0=0

history_l2 = model_l2.fit(X_train, y_train, validation_data=(X_valid,

val_auc_l2 = model_l2.evaluate(X_valid, y_valid)[1]

```

Epoch 1/30

817/817  **1s** 959us/step - AUC: 0.8226 - loss: 0.4404
- val_AUC: 0.8983 - val_loss: 0.3795 - learning_rate: 0.0100

Epoch 2/30

817/817  **1s** 796us/step - AUC: 0.9086 - loss: 0.3671
- val_AUC: 0.9039 - val_loss: 0.3654 - learning_rate: 0.0089

Epoch 3/30

817/817  **1s** 790us/step - AUC: 0.9145 - loss: 0.3539
- val_AUC: 0.9073 - val_loss: 0.3554 - learning_rate: 0.0079

Epoch 4/30

817/817  **1s** 795us/step - AUC: 0.9180 - loss: 0.3441
- val_AUC: 0.9090 - val_loss: 0.3476 - learning_rate: 0.0071

Epoch 5/30

817/817  **1s** 795us/step - AUC: 0.9202 - loss: 0.3363
- val_AUC: 0.9108 - val_loss: 0.3412 - learning_rate: 0.0063

Epoch 6/30

817/817  **1s** 821us/step - AUC: 0.9221 - loss: 0.3299
- val_AUC: 0.9116 - val_loss: 0.3359 - learning_rate: 0.0056

Epoch 7/30

817/817  **1s** 794us/step - AUC: 0.9236 - loss: 0.3245
- val_AUC: 0.9126 - val_loss: 0.3315 - learning_rate: 0.0050

Epoch 8/30

817/817  **1s** 794us/step - AUC: 0.9247 - loss: 0.3199
- val_AUC: 0.9136 - val_loss: 0.3277 - learning_rate: 0.0045

Epoch 9/30

817/817  **1s** 792us/step - AUC: 0.9258 - loss: 0.3159
- val_AUC: 0.9142 - val_loss: 0.3245 - learning_rate: 0.0040

Epoch 10/30

817/817  **1s** 793us/step - AUC: 0.9267 - loss: 0.3125
- val_AUC: 0.9145 - val_loss: 0.3217 - learning_rate: 0.0035

Epoch 11/30

817/817  **1s** 795us/step - AUC: 0.9275 - loss: 0.3095
- val_AUC: 0.9151 - val_loss: 0.3194 - learning_rate: 0.0032

Epoch 12/30

817/817  **1s** 826us/step - AUC: 0.9281 - loss: 0.3069
- val_AUC: 0.9151 - val_loss: 0.3173 - learning_rate: 0.0028

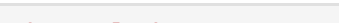
Epoch 13/30

817/817  **1s** 807us/step - AUC: 0.9288 - loss: 0.3046
- val_AUC: 0.9156 - val_loss: 0.3155 - learning_rate: 0.0025

Epoch 14/30

817/817  **1s** 791us/step - AUC: 0.9296 - loss: 0.3026

```

- val_AUC: 0.9158 - val_loss: 0.3140 - learning_rate: 0.0022
Epoch 15/30
817/817  1s 792us/step - AUC: 0.9301 - loss: 0.3008
- val_AUC: 0.9161 - val_loss: 0.3126 - learning_rate: 0.0020
Epoch 16/30
817/817  1s 800us/step - AUC: 0.9305 - loss: 0.2992
- val_AUC: 0.9165 - val_loss: 0.3115 - learning_rate: 0.0018
Epoch 17/30
817/817  1s 819us/step - AUC: 0.9310 - loss: 0.2977
- val_AUC: 0.9167 - val_loss: 0.3104 - learning_rate: 0.0016
Epoch 18/30
817/817  1s 820us/step - AUC: 0.9313 - loss: 0.2965
- val_AUC: 0.9163 - val_loss: 0.3095 - learning_rate: 0.0014
Epoch 19/30
817/817  1s 791us/step - AUC: 0.9317 - loss: 0.2953
- val_AUC: 0.9164 - val_loss: 0.3087 - learning_rate: 0.0013
Epoch 20/30
817/817  1s 789us/step - AUC: 0.9320 - loss: 0.2943
- val_AUC: 0.9166 - val_loss: 0.3081 - learning_rate: 0.0011
Epoch 21/30
817/817  1s 789us/step - AUC: 0.9322 - loss: 0.2934
- val_AUC: 0.9169 - val_loss: 0.3075 - learning_rate: 0.0010
Epoch 22/30
817/817  1s 793us/step - AUC: 0.9325 - loss: 0.2926
- val_AUC: 0.9170 - val_loss: 0.3069 - learning_rate: 8.9125e-04
Epoch 23/30
817/817  1s 816us/step - AUC: 0.9327 - loss: 0.2919
- val_AUC: 0.9171 - val_loss: 0.3065 - learning_rate: 7.9433e-04
Epoch 24/30
817/817  1s 796us/step - AUC: 0.9329 - loss: 0.2912
- val_AUC: 0.9165 - val_loss: 0.3061 - learning_rate: 7.0795e-04
Epoch 25/30
817/817  1s 793us/step - AUC: 0.9331 - loss: 0.2906
- val_AUC: 0.9166 - val_loss: 0.3058 - learning_rate: 6.3096e-04
Epoch 26/30
817/817  1s 792us/step - AUC: 0.9333 - loss: 0.2901
- val_AUC: 0.9164 - val_loss: 0.3055 - learning_rate: 5.6234e-04
Epoch 27/30
817/817  1s 793us/step - AUC: 0.9334 - loss: 0.2896
- val_AUC: 0.9166 - val_loss: 0.3052 - learning_rate: 5.0119e-04
Epoch 28/30
817/817  1s 822us/step - AUC: 0.9336 - loss: 0.2892
- val_AUC: 0.9167 - val_loss: 0.3050 - learning_rate: 4.4668e-04
Epoch 29/30
817/817  1s 793us/step - AUC: 0.9337 - loss: 0.2888
- val_AUC: 0.9166 - val_loss: 0.3048 - learning_rate: 3.9811e-04
Epoch 30/30
817/817  1s 789us/step - AUC: 0.9339 - loss: 0.2885
- val_AUC: 0.9167 - val_loss: 0.3046 - learning_rate: 3.5481e-04
205/205  0s 497us/step - AUC: 0.9159 - loss: 0.3102

```

```
In [ ]: print("The validation AUC of the model with l2 regularization: ", val_
```

The validation AUC of the model with ℓ_2 regularization: 0.9166568517684937

The validation AUC of the model (b): 0.9136109352111816

The validation AUC of the model with ℓ_2 regularization: 0.9166568517684937

Comparing the model in (b) with the model using ℓ_2 regularization technique, it seems like the validation AUC is slightly higher for the model using ℓ_2 regularization, which tells the model with ℓ_2 regularization is better than the model in (b).

```
In [ ]: # (iv) dropout with probability 0.02

reset_session()

model_drop = tf.keras.models.Sequential([
    layers.Dense(100, activation='swish', kernel_initializer='he_normal'),
    layers.Dropout(0.02),
    layers.Dense(100, activation='swish', kernel_initializer='he_normal'),
    layers.Dropout(0.02),
    layers.Dense(100, activation='swish', kernel_initializer='he_normal'),
    layers.Dropout(0.02),
    layers.Dense(100, activation='swish', kernel_initializer='he_normal'),
    layers.Dropout(0.02),
    layers.Dense(1, activation='sigmoid')
])

opt = optimizers.SGD(learning_rate=0.01, momentum=0.9, nesterov=True)
model_drop.compile(optimizer=opt, loss="binary_crossentropy", metrics=[

lr_scheduler = callbacks.LearningRateScheduler(exponential_decay(lr0=0

history_drop = model_drop.fit(X_train, y_train, validation_data=(X_val

val_auc_drop = model_drop.evaluate(X_valid, y_valid)[1])
```

Epoch 1/30

817/817 ————— 1s 996us/step – AUC: 0.8307 – loss: 0.2782
– val_AUC: 0.8949 – val_loss: 0.2280 – learning_rate: 0.0100

Epoch 2/30

817/817 ————— 1s 867us/step – AUC: 0.9036 – loss: 0.2211
– val_AUC: 0.9035 – val_loss: 0.2206 – learning_rate: 0.0089

Epoch 3/30

817/817 ————— 1s 866us/step – AUC: 0.9099 – loss: 0.2154
– val_AUC: 0.9080 – val_loss: 0.2162 – learning_rate: 0.0079

Epoch 4/30

817/817 ————— 1s 862us/step – AUC: 0.9146 – loss: 0.2114
– val_AUC: 0.9099 – val_loss: 0.2136 – learning_rate: 0.0071

Epoch 5/30

817/817 ————— 1s 864us/step – AUC: 0.9182 – loss: 0.2076

- val_AUC: 0.9121 - val_loss: 0.2114 - learning_rate: 0.0063
Epoch 6/30
817/817  1s 895us/step - AUC: 0.9203 - loss: 0.2054
- val_AUC: 0.9132 - val_loss: 0.2104 - learning_rate: 0.0056
Epoch 7/30
817/817  1s 867us/step - AUC: 0.9206 - loss: 0.2049
- val_AUC: 0.9140 - val_loss: 0.2090 - learning_rate: 0.0050
Epoch 8/30
817/817  1s 875us/step - AUC: 0.9242 - loss: 0.2011
- val_AUC: 0.9147 - val_loss: 0.2085 - learning_rate: 0.0045
Epoch 9/30
817/817  1s 859us/step - AUC: 0.9252 - loss: 0.2005
- val_AUC: 0.9149 - val_loss: 0.2083 - learning_rate: 0.0040
Epoch 10/30
817/817  1s 863us/step - AUC: 0.9270 - loss: 0.1986
- val_AUC: 0.9158 - val_loss: 0.2078 - learning_rate: 0.0035
Epoch 11/30
817/817  1s 889us/step - AUC: 0.9274 - loss: 0.1979
- val_AUC: 0.9158 - val_loss: 0.2071 - learning_rate: 0.0032
Epoch 12/30
817/817  1s 862us/step - AUC: 0.9282 - loss: 0.1966
- val_AUC: 0.9159 - val_loss: 0.2072 - learning_rate: 0.0028
Epoch 13/30
817/817  1s 865us/step - AUC: 0.9288 - loss: 0.1959
- val_AUC: 0.9165 - val_loss: 0.2068 - learning_rate: 0.0025
Epoch 14/30
817/817  1s 862us/step - AUC: 0.9294 - loss: 0.1955
- val_AUC: 0.9165 - val_loss: 0.2067 - learning_rate: 0.0022
Epoch 15/30
817/817  1s 860us/step - AUC: 0.9298 - loss: 0.1949
- val_AUC: 0.9163 - val_loss: 0.2065 - learning_rate: 0.0020
Epoch 16/30
817/817  1s 889us/step - AUC: 0.9301 - loss: 0.1944
- val_AUC: 0.9167 - val_loss: 0.2066 - learning_rate: 0.0018
Epoch 17/30
817/817  1s 866us/step - AUC: 0.9324 - loss: 0.1923
- val_AUC: 0.9168 - val_loss: 0.2064 - learning_rate: 0.0016
Epoch 18/30
817/817  1s 862us/step - AUC: 0.9317 - loss: 0.1926
- val_AUC: 0.9168 - val_loss: 0.2064 - learning_rate: 0.0014
Epoch 19/30
817/817  1s 858us/step - AUC: 0.9324 - loss: 0.1916
- val_AUC: 0.9169 - val_loss: 0.2064 - learning_rate: 0.0013
Epoch 20/30
817/817  1s 860us/step - AUC: 0.9327 - loss: 0.1912
- val_AUC: 0.9168 - val_loss: 0.2064 - learning_rate: 0.0011
Epoch 21/30
817/817  1s 877us/step - AUC: 0.9339 - loss: 0.1901
- val_AUC: 0.9172 - val_loss: 0.2064 - learning_rate: 0.0010
Epoch 22/30
817/817  1s 864us/step - AUC: 0.9335 - loss: 0.1909
- val_AUC: 0.9170 - val_loss: 0.2064 - learning_rate: 8.9125e-04

```

Epoch 23/30
817/817 ————— 1s 855us/step - AUC: 0.9339 - loss: 0.1901
- val_AUC: 0.9171 - val_loss: 0.2063 - learning_rate: 7.9433e-04
Epoch 24/30
817/817 ————— 1s 865us/step - AUC: 0.9336 - loss: 0.1904
- val_AUC: 0.9170 - val_loss: 0.2064 - learning_rate: 7.0795e-04
Epoch 25/30
817/817 ————— 1s 883us/step - AUC: 0.9350 - loss: 0.1887
- val_AUC: 0.9167 - val_loss: 0.2065 - learning_rate: 6.3096e-04
Epoch 26/30
817/817 ————— 1s 859us/step - AUC: 0.9350 - loss: 0.1890
- val_AUC: 0.9168 - val_loss: 0.2067 - learning_rate: 5.6234e-04
Epoch 27/30
817/817 ————— 1s 858us/step - AUC: 0.9351 - loss: 0.1881
- val_AUC: 0.9167 - val_loss: 0.2069 - learning_rate: 5.0119e-04
Epoch 28/30
817/817 ————— 1s 881us/step - AUC: 0.9337 - loss: 0.1898
- val_AUC: 0.9169 - val_loss: 0.2070 - learning_rate: 4.4668e-04
Epoch 29/30
817/817 ————— 1s 859us/step - AUC: 0.9353 - loss: 0.1886
- val_AUC: 0.9170 - val_loss: 0.2070 - learning_rate: 3.9811e-04
Epoch 30/30
817/817 ————— 1s 886us/step - AUC: 0.9345 - loss: 0.1890
- val_AUC: 0.9167 - val_loss: 0.2071 - learning_rate: 3.5481e-04
205/205 ————— 0s 489us/step - AUC: 0.9162 - loss: 0.2134

```

```
In [ ]: print("The validation AUC of the model with dropout with probability 0
```

```
The validation AUC of the model with dropout with probability 0.02: 0.9166733026504517
```

The validation AUC of the model (b): 0.9136109352111816

The validation AUC of the model with dropout with probability 0.02:
0.9166733026504517

Comparing the model in (b) with the model using dropout with probability 0.02 regularization technique, it seems like the validation AUC is slightly higher for the model using dropout with probability 0.02, which tells the model with dropout with probability 0.02 is better than the model in (b).

```
In [ ]: # (v) $ell_2$ regularization and early stopping both as above

reset_session()

l2_reg = tf.keras.regularizers.l2(0.0002)

model_l2es = tf.keras.models.Sequential([
    layers.Dense(100, activation='swish', kernel_initializer='he_normal',
    layers.Dense(100, activation='swish', kernel_initializer='he_normal',
    layers.Dense(100, activation='swish', kernel_initializer='he_normal',

```

```

layers.Dense(100, activation='swish', kernel_initializer="he_normal")
layers.Dense(1, activation="sigmoid")
])

opt = optimizers.SGD(learning_rate=0.01, momentum=0.9, nesterov=True)
model_l2es.compile(optimizer=opt, loss="binary_crossentropy", metrics=

lr_scheduler = callbacks.LearningRateScheduler(exponential_decay(lr0=0

early_stop = callbacks.EarlyStopping(monitor='val_auc', patience=10, m

history_l2es = model_l2es.fit(X_train, y_train, validation_data=(X_val

val_auc_l2es = model_l2es.evaluate(X_valid, y_valid)[1]

```

Epoch 1/30

817/817 ————— 1s 942us/step - AUC: 0.8274 - loss: 0.4310
 - val_AUC: 0.8985 - val_loss: 0.3784 - learning_rate: 0.0100

Epoch 2/30

817/817 ————— 1s 803us/step - AUC: 0.9069 - loss: 0.3685
 - val_AUC: 0.9042 - val_loss: 0.3648 - learning_rate: 0.0089

Epoch 3/30

817/817 ————— 1s 792us/step - AUC: 0.9127 - loss: 0.3554
 - val_AUC: 0.9073 - val_loss: 0.3550 - learning_rate: 0.0079

Epoch 4/30

817/817 ————— 1s 795us/step - AUC: 0.9163 - loss: 0.3458
 - val_AUC: 0.9093 - val_loss: 0.3472 - learning_rate: 0.0071

Epoch 5/30

817/817 ————— 1s 796us/step - AUC: 0.9188 - loss: 0.3381
 - val_AUC: 0.9110 - val_loss: 0.3408 - learning_rate: 0.0063

Epoch 6/30

817/817 ————— 1s 793us/step - AUC: 0.9207 - loss: 0.3316
 - val_AUC: 0.9123 - val_loss: 0.3354 - learning_rate: 0.0056

Epoch 7/30

817/817 ————— 1s 798us/step - AUC: 0.9222 - loss: 0.3261
 - val_AUC: 0.9134 - val_loss: 0.3308 - learning_rate: 0.0050

Epoch 8/30

817/817 ————— 1s 831us/step - AUC: 0.9235 - loss: 0.3215
 - val_AUC: 0.9142 - val_loss: 0.3270 - learning_rate: 0.0045

Epoch 9/30

817/817 ————— 1s 795us/step - AUC: 0.9247 - loss: 0.3175
 - val_AUC: 0.9146 - val_loss: 0.3237 - learning_rate: 0.0040

Epoch 10/30

817/817 ————— 1s 798us/step - AUC: 0.9256 - loss: 0.3140
 - val_AUC: 0.9154 - val_loss: 0.3209 - learning_rate: 0.0035

Epoch 11/30

817/817 ————— 1s 794us/step - AUC: 0.9265 - loss: 0.3110
 - val_AUC: 0.9156 - val_loss: 0.3185 - learning_rate: 0.0032

Epoch 12/30

817/817 ————— 1s 795us/step - AUC: 0.9271 - loss: 0.3083
 - val_AUC: 0.9159 - val_loss: 0.3164 - learning_rate: 0.0028

Epoch 13/30

817/817 ————— 1s 795us/step - AUC: 0.9277 - loss: 0.3060

- val_AUC: 0.9161 - val_loss: 0.3146 - learning_rate: 0.0025
Epoch 14/30
817/817  1s 824us/step - AUC: 0.9283 - loss: 0.3040
- val_AUC: 0.9161 - val_loss: 0.3131 - learning_rate: 0.0022
Epoch 15/30
817/817  1s 801us/step - AUC: 0.9288 - loss: 0.3022
- val_AUC: 0.9165 - val_loss: 0.3117 - learning_rate: 0.0020
Epoch 16/30
817/817  1s 794us/step - AUC: 0.9292 - loss: 0.3006
- val_AUC: 0.9166 - val_loss: 0.3105 - learning_rate: 0.0018
Epoch 17/30
817/817  1s 793us/step - AUC: 0.9296 - loss: 0.2992
- val_AUC: 0.9169 - val_loss: 0.3095 - learning_rate: 0.0016
Epoch 18/30
817/817  1s 794us/step - AUC: 0.9301 - loss: 0.2979
- val_AUC: 0.9168 - val_loss: 0.3086 - learning_rate: 0.0014
Epoch 19/30
817/817  1s 847us/step - AUC: 0.9303 - loss: 0.2968
- val_AUC: 0.9169 - val_loss: 0.3078 - learning_rate: 0.0013
Epoch 20/30
817/817  1s 800us/step - AUC: 0.9306 - loss: 0.2958
- val_AUC: 0.9171 - val_loss: 0.3071 - learning_rate: 0.0011
Epoch 21/30
817/817  1s 792us/step - AUC: 0.9309 - loss: 0.2949
- val_AUC: 0.9172 - val_loss: 0.3065 - learning_rate: 0.0010
Epoch 22/30
817/817  1s 797us/step - AUC: 0.9311 - loss: 0.2941
- val_AUC: 0.9175 - val_loss: 0.3060 - learning_rate: 8.9125e-04
Epoch 23/30
817/817  1s 797us/step - AUC: 0.9314 - loss: 0.2933
- val_AUC: 0.9175 - val_loss: 0.3055 - learning_rate: 7.9433e-04
Epoch 24/30
817/817  1s 827us/step - AUC: 0.9316 - loss: 0.2927
- val_AUC: 0.9176 - val_loss: 0.3051 - learning_rate: 7.0795e-04
Epoch 25/30
817/817  1s 791us/step - AUC: 0.9317 - loss: 0.2921
- val_AUC: 0.9177 - val_loss: 0.3048 - learning_rate: 6.3096e-04
Epoch 26/30
817/817  1s 793us/step - AUC: 0.9318 - loss: 0.2916
- val_AUC: 0.9178 - val_loss: 0.3045 - learning_rate: 5.6234e-04
Epoch 27/30
817/817  1s 791us/step - AUC: 0.9320 - loss: 0.2911
- val_AUC: 0.9174 - val_loss: 0.3042 - learning_rate: 5.0119e-04
Epoch 28/30
817/817  1s 825us/step - AUC: 0.9321 - loss: 0.2907
- val_AUC: 0.9175 - val_loss: 0.3040 - learning_rate: 4.4668e-04
Epoch 29/30
817/817  1s 794us/step - AUC: 0.9324 - loss: 0.2903
- val_AUC: 0.9176 - val_loss: 0.3038 - learning_rate: 3.9811e-04
Epoch 30/30
817/817  1s 796us/step - AUC: 0.9325 - loss: 0.2900
- val_AUC: 0.9176 - val_loss: 0.3036 - learning_rate: 3.5481e-04

205/205 ————— **0s** 495us/step – AUC: 0.9179 – loss: 0.3096

```
In [ ]: print("The validation AUC of the model with l2 regularization and early stopping: 0.9175518751144409")
```

The validation AUC of the model with l2 regularization and early stopping: 0.9175518751144409

The validation AUC of the model (b): 0.9136109352111816

The validation AUC of the model with ℓ_2 regularization and early stopping: 0.9175518751144409

Comparing the model in (b) with the model using ℓ_2 regularization and early stopping technique, it seems like the validation AUC is higher for the model with ℓ_2 regularization and early stopping, which tells the model with ℓ_2 regularization and early stopping is better than the model in (b).

```
In [ ]: # (vi) batch normalization and dropout both as above

reset_session()

model_bn_drop = tf.keras.models.Sequential([
    layers.Dense(100, activation='swish', kernel_initializer='he_normal'),
    layers.BatchNormalization(),
    layers.Dropout(0.02),
    layers.Dense(100, activation='swish', kernel_initializer='he_normal'),
    layers.BatchNormalization(),
    layers.Dropout(0.02),
    layers.Dense(100, activation='swish', kernel_initializer='he_normal'),
    layers.BatchNormalization(),
    layers.Dropout(0.02),
    layers.Dense(100, activation='swish', kernel_initializer='he_normal'),
    layers.BatchNormalization(),
    layers.Dropout(0.02),
    layers.Dense(1, activation='sigmoid')
])

opt = optimizers.SGD(learning_rate=0.01, momentum=0.9, nesterov=True)
model_bn_drop.compile(optimizer=opt, loss="binary_crossentropy", metrics=['accuracy'])

lr_scheduler = callbacks.LearningRateScheduler(exponential_decay(lr0=0.01, decay_rate=0.1, decay_steps=10000))

history_bn_drop = model_bn_drop.fit(X_train, y_train, validation_data=(X_valid, y_valid), callbacks=[lr_scheduler])

val_auc_bn_drop = model_bn_drop.evaluate(X_valid, y_valid)[1]
```

Epoch 1/30

817/817 ————— **2s** 1ms/step – AUC: 0.8039 – loss: 0.3234 – val_AUC: 0.8962 – val_loss: 0.2295 – learning_rate: 0.0100

Epoch 2/30

817/817 ————— **1s** 1ms/step – AUC: 0.9071 – loss: 0.2196 – val_AUC: 0.9011 – val_loss: 0.2290 – learning_rate: 0.0089

Epoch 3/30
817/817 ————— **1s** 1ms/step - AUC: 0.9168 - loss: 0.2095 -
val_AUC: 0.9020 - val_loss: 0.2287 - learning_rate: 0.0079
Epoch 4/30
817/817 ————— **1s** 1ms/step - AUC: 0.9266 - loss: 0.1991 -
val_AUC: 0.8997 - val_loss: 0.2329 - learning_rate: 0.0071
Epoch 5/30
817/817 ————— **1s** 1ms/step - AUC: 0.9313 - loss: 0.1927 -
val_AUC: 0.8995 - val_loss: 0.2381 - learning_rate: 0.0063
Epoch 6/30
817/817 ————— **1s** 1ms/step - AUC: 0.9360 - loss: 0.1864 -
val_AUC: 0.9007 - val_loss: 0.2365 - learning_rate: 0.0056
Epoch 7/30
817/817 ————— **1s** 1ms/step - AUC: 0.9432 - loss: 0.1775 -
val_AUC: 0.9012 - val_loss: 0.2398 - learning_rate: 0.0050
Epoch 8/30
817/817 ————— **1s** 1ms/step - AUC: 0.9476 - loss: 0.1702 -
val_AUC: 0.8963 - val_loss: 0.2469 - learning_rate: 0.0045
Epoch 9/30
817/817 ————— **1s** 1ms/step - AUC: 0.9506 - loss: 0.1646 -
val_AUC: 0.8948 - val_loss: 0.2510 - learning_rate: 0.0040
Epoch 10/30
817/817 ————— **1s** 1ms/step - AUC: 0.9557 - loss: 0.1577 -
val_AUC: 0.8948 - val_loss: 0.2554 - learning_rate: 0.0035
Epoch 11/30
817/817 ————— **1s** 1ms/step - AUC: 0.9605 - loss: 0.1493 -
val_AUC: 0.8928 - val_loss: 0.2604 - learning_rate: 0.0032
Epoch 12/30
817/817 ————— **1s** 1ms/step - AUC: 0.9659 - loss: 0.1398 -
val_AUC: 0.8901 - val_loss: 0.2671 - learning_rate: 0.0028
Epoch 13/30
817/817 ————— **1s** 1ms/step - AUC: 0.9667 - loss: 0.1359 -
val_AUC: 0.8883 - val_loss: 0.2688 - learning_rate: 0.0025
Epoch 14/30
817/817 ————— **1s** 1ms/step - AUC: 0.9695 - loss: 0.1325 -
val_AUC: 0.8872 - val_loss: 0.2697 - learning_rate: 0.0022
Epoch 15/30
817/817 ————— **1s** 1ms/step - AUC: 0.9727 - loss: 0.1250 -
val_AUC: 0.8877 - val_loss: 0.2732 - learning_rate: 0.0020
Epoch 16/30
817/817 ————— **1s** 1ms/step - AUC: 0.9746 - loss: 0.1195 -
val_AUC: 0.8877 - val_loss: 0.2761 - learning_rate: 0.0018
Epoch 17/30
817/817 ————— **1s** 1ms/step - AUC: 0.9757 - loss: 0.1181 -
val_AUC: 0.8849 - val_loss: 0.2818 - learning_rate: 0.0016
Epoch 18/30
817/817 ————— **1s** 1ms/step - AUC: 0.9800 - loss: 0.1089 -
val_AUC: 0.8835 - val_loss: 0.2854 - learning_rate: 0.0014
Epoch 19/30
817/817 ————— **1s** 1ms/step - AUC: 0.9790 - loss: 0.1104 -
val_AUC: 0.8849 - val_loss: 0.2876 - learning_rate: 0.0013
Epoch 20/30

```

817/817 ----- 1s 1ms/step - AUC: 0.9797 - loss: 0.1072 -
val_AUC: 0.8810 - val_loss: 0.2941 - learning_rate: 0.0011
Epoch 21/30
817/817 ----- 1s 1ms/step - AUC: 0.9806 - loss: 0.1050 -
val_AUC: 0.8811 - val_loss: 0.2931 - learning_rate: 0.0010
Epoch 22/30
817/817 ----- 1s 1ms/step - AUC: 0.9831 - loss: 0.0996 -
val_AUC: 0.8811 - val_loss: 0.2958 - learning_rate: 8.9125e-04
Epoch 23/30
817/817 ----- 1s 1ms/step - AUC: 0.9826 - loss: 0.1010 -
val_AUC: 0.8762 - val_loss: 0.2999 - learning_rate: 7.9433e-04
Epoch 24/30
817/817 ----- 1s 1ms/step - AUC: 0.9830 - loss: 0.0966 -
val_AUC: 0.8786 - val_loss: 0.2991 - learning_rate: 7.0795e-04
Epoch 25/30
817/817 ----- 1s 1ms/step - AUC: 0.9842 - loss: 0.0949 -
val_AUC: 0.8786 - val_loss: 0.3021 - learning_rate: 6.3096e-04
Epoch 26/30
817/817 ----- 1s 1ms/step - AUC: 0.9860 - loss: 0.0911 -
val_AUC: 0.8809 - val_loss: 0.3020 - learning_rate: 5.6234e-04
Epoch 27/30
817/817 ----- 1s 1ms/step - AUC: 0.9866 - loss: 0.0906 -
val_AUC: 0.8790 - val_loss: 0.3056 - learning_rate: 5.0119e-04
Epoch 28/30
817/817 ----- 1s 1ms/step - AUC: 0.9871 - loss: 0.0875 -
val_AUC: 0.8790 - val_loss: 0.3068 - learning_rate: 4.4668e-04
Epoch 29/30
817/817 ----- 1s 1ms/step - AUC: 0.9866 - loss: 0.0899 -
val_AUC: 0.8754 - val_loss: 0.3089 - learning_rate: 3.9811e-04
Epoch 30/30
817/817 ----- 1s 1ms/step - AUC: 0.9860 - loss: 0.0900 -
val_AUC: 0.8779 - val_loss: 0.3098 - learning_rate: 3.5481e-04
205/205 ----- 0s 517us/step - AUC: 0.8656 - loss: 0.3324

```

```
In [ ]: print("The validation AUC of the model with both batch normalization a
```

The validation AUC of the model with both batch normalization and dropout: 0.8779004812240601

The validation AUC of the model (b): 0.9136109352111816

The validation AUC of the model with both batch normalization and dropout:
0.8779004812240601

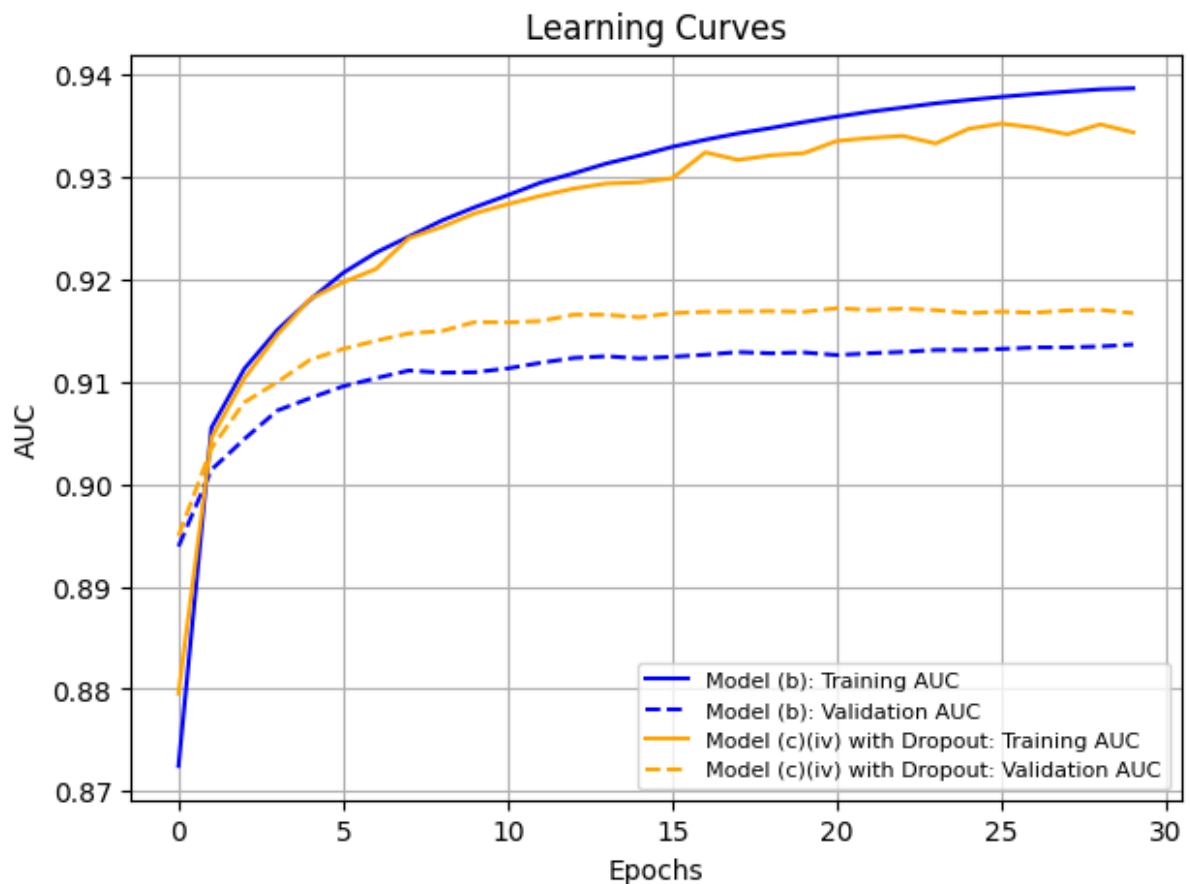
Comparing the model in (b) with the model using both batch normalization and dropout regularization technique, it seems like the validation AUC is higher for model (b), which tells the model with both batch normalization and dropout is NOT better than the model in (b).

(d) [1 mark]

For the dropout model in (c)(iv) determine whether or not it is overfitting less than the model in (b).

[Add your solution here]

```
In [ ]: plt.figure(figsize=(7,5))
plt.plot(history_b.history['AUC'], color='blue', label='Model (b): Tra
plt.plot(history_b.history['val_AUC'], linestyle='--', color='blue', l
plt.plot(history_drop.history['AUC'], color='orange', label='Model (c)
plt.plot(history_drop.history['val_AUC'], linestyle='--', color='orang
plt.title("Learning Curves")
plt.xlabel('Epochs')
plt.ylabel('AUC')
plt.legend(fontsize=8)
plt.grid(True)
plt.show()
```



The dropout model in (c)(iv) is less overfitting than the model in (b) because when we can see the space/distance between the training AUC and the validation AUC curve for each model, the dropout model in (c)(iv) is converging more and has a smaller gap compared to the model in (b).

(e) [1 mark]

Of the models in (b) and (c), one would now choose the best model according to the performance metric (validation AUC) to evaluate on the test set. But instead, evaluate the model in (c)(v) on the test set in terms of the AUC and confusion matrix (regardless of whether it is the best model given your results).

[Add your solution here]

```
In [ ]: test_loss, test_auc = model_l2es.evaluate(X_test, y_test)
print("Model in (c)(v) - AUC of the test set:", test_auc)
print("Model in (c)(v) - Loss of the test set:", test_loss)
```

256/256 ————— 0s 506us/step - AUC: 0.9246 - loss: 0.3035
 Model in (c)(v) - AUC of the test set: 0.9299761056900024
 Model in (c)(v) - Loss of the test set: 0.3043237030506134

```
In [ ]: from sklearn.metrics import confusion_matrix

y_test_pred = (model_l2es.predict(X_test) > 0.5).astype(int)
confusion_mat = confusion_matrix(y_test, y_test_pred)
print("Model in (c)(v) - Confusion Matrix:")
print(confusion_mat)

# [0][0] true negatives
# [0][1] false positives
# [1][0] false negatives
# [1][1] true positives
```

256/256 ————— 0s 419us/step
 Model in (c)(v) - Confusion Matrix:
 [[6947 219]
 [545 458]]

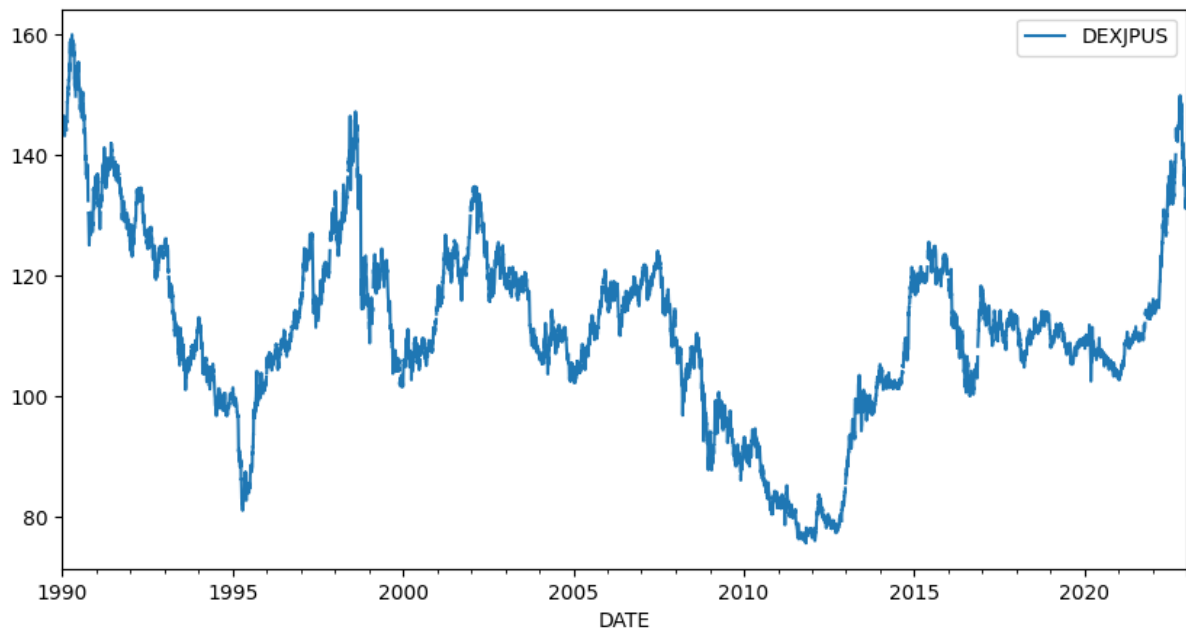
3. Time series using machine learning [14 marks]

Obtain daily values of the [Japan/U.S. Foreign Exchange Rate \(DEXJPUS\)](#) starting from Jan 1, 1990, to Jan 1, 2023, from FRED. This can be obtained using the code below or you can download the data as a csv file from [Canvas](#).

```
In [ ]: import pandas as pd
import pandas_datareader as pdr
from datetime import datetime
data = pdr.get_data_fred('DEXJPUS', datetime(1990,1,1),datetime(2023,1
```

```
In [ ]: data.plot(figsize=(10,5))
```

```
Out[ ]: <Axes: xlabel='DATE'>
```



(a) [2 marks]

Create a training set (before 2010), a validation set (Jan 2010 to Dec 2015), and a test set (the rest of the data). Turn the time series data into a supervised learning dataset where the features are the value of the exchange rate in the last 10 days inclusive of the current day, and the target is the value of the exchange rate in the next day.

[Add your solution here]

```
In [ ]: data.dropna(inplace=True)
```

```
train = data[:'2009-12-31']
validation = data['2010-01-01':'2015-12-31']
test = data['2016-01-01':]
```

```
In [ ]: def ts_split(ts, feature_steps=10, target_steps=1):
    n_obs = len(ts) - feature_steps - target_steps + 1
    X = np.array([ts[index:index + feature_steps].values for index in
    y = np.array([ts[index + feature_steps:index + feature_steps + tar
    return X, y
```

```
In [ ]: X_train, y_train = ts_split(train['DEXJPUS'])
X_valid, y_valid = ts_split(validation['DEXJPUS'])
X_test, y_test = ts_split(test['DEXJPUS'])

y_train = y_train.flatten()
y_valid = y_valid.flatten()
y_test = y_test.flatten()
```

(b) [3 marks]

Fit a random forest regressor to predict the value of the exchange rate in the next day. Using the test set, report the mean squared error and the accuracy for the movement direction.

Hint: You can calculate the accuracy of the movement direction by determining what the actual movement direction is and comparing it to the movement direction corresponding to the predicted value of the exchange rate. For instance, the movement direction of the test set `X_test` and `y_test` where a strictly up movement is `True` can be computed as follows.

```
In [ ]: movement_test = X_test[:, -1] < y_test.ravel()
```

[Add your solution here]

```
In [ ]: from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, accuracy_score

reset_session()

rf = RandomForestRegressor(random_state=42)
rf.fit(X_train, y_train)

y_test_pred = rf.predict(X_test)

mse = mean_squared_error(y_test, y_test_pred)
print('The Mean Squared Error of the Random Forest Regressor: ', mse)

movement_test = X_test[:, -1] < y_test.ravel()
movement_pred = X_test[:, -1] < y_test_pred.ravel()
movement_accuracy = accuracy_score(movement_test, movement_pred)
print('The Accuracy for the Movement direction: ', movement_accuracy)
```

The Mean Squared Error of the Random Forest Regressor: 0.5306154127633844

The Accuracy for the Movement direction: 0.5215889464594128

(c) [4 marks]

Repeat (b), but now fit a deep RNN with 2 recurrent layers of 20 and 20 neurons, and an output layer which is 1 dense neuron. Use 100 epochs and the Nadam optimizer. Comment on the result and the learning curve (the validation set is used for the learning curve).

[Add your solution here]

```
In [ ]: from tensorflow.keras.layers import Dense, SimpleRNN

reset_session()

model = tf.keras.models.Sequential([SimpleRNN(20, return_sequences=True),
                                     SimpleRNN(20),
                                     Dense(1)])

model.compile(loss="mse", optimizer="nadam")

history_1 = model.fit(X_train[..., np.newaxis], y_train, epochs=100,
                      validation_data = (X_valid[..., np.newaxis], y_v

y_test_pred = model.predict(X_test[..., np.newaxis])
mse_rnn = mean_squared_error(y_test, y_test_pred)
print('The Mean Squared Error of the Simple RNN model: ', mse_rnn)
```

/Users/eunicehan/.pyenv/versions/3.10.11/lib/python3.10/site-packages/k
 eras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input_shap
 e`/`input_dim` argument to a layer. When using Sequential models, pref
 er using an `Input(shape)` object as the first layer in the model instea
 d.

super().__init__(**kwargs)

55/55 ————— 0s 2ms/step

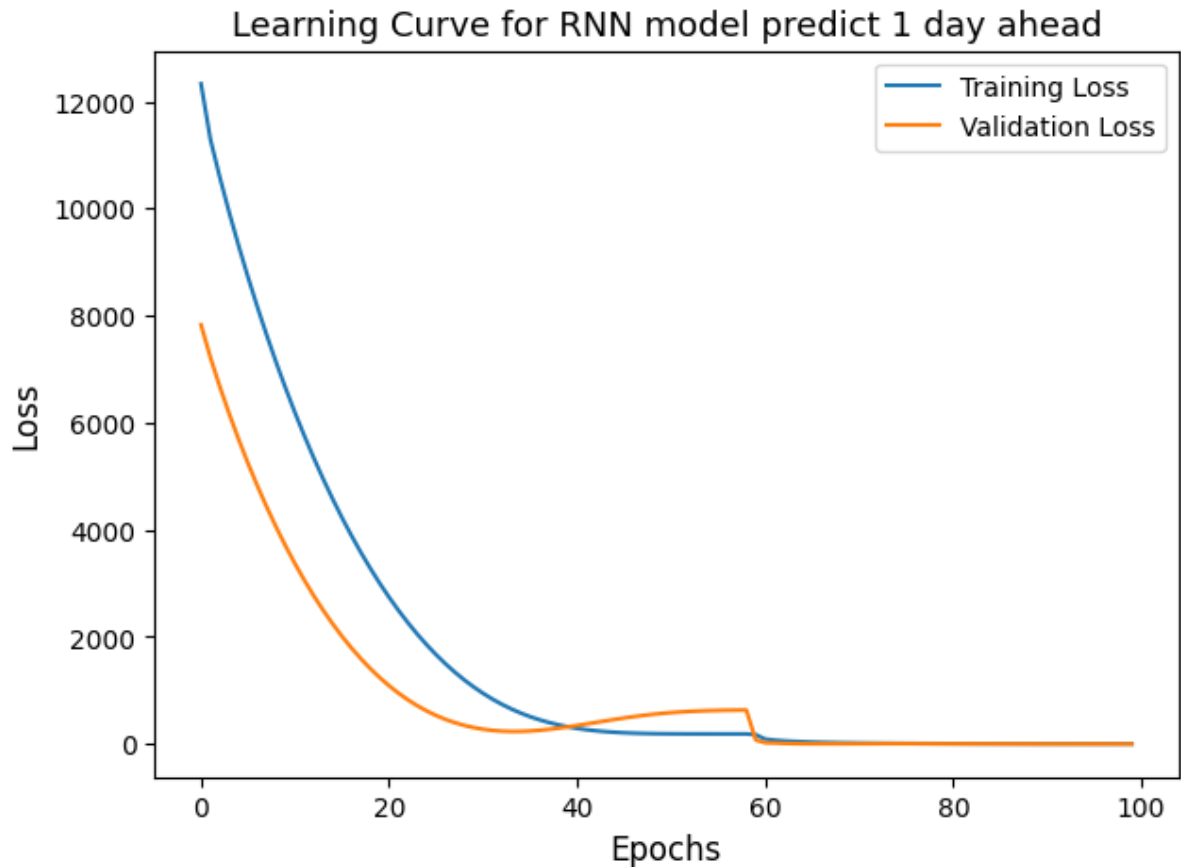
The Mean Squared Error of the Simple RNN model: 0.571463096905751

```
In [ ]: movement_pred = X_test[:, -1] < y_test_pred.ravel()
movement_accuracy = accuracy_score(movement_test, movement_pred)
print('The Accuracy of the Movement direction: ', movement_accuracy)
```

The Accuracy of the Movement direction: 0.5008635578583766

Looking into the results, it look likes the deep RNN model got mean squared error: 0.571463096905751, and accuracy: 0.5008635578583766. Comparing it with model in (b) - Random Forest Regressor (mean squared error: 0.5306154127633844 accuracy: 0.5215889464594128), it seems like model in (b) is better as the mean squared error is lower while the accuracy of the movement direction is higher.

```
In [ ]: plt.figure(figsize=(7,5))
plt.plot(history_1.history['loss'], label='Training Loss')
plt.plot(history_1.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs', fontsize=12)
plt.ylabel('Loss', fontsize=12)
plt.title('Learning Curve for RNN model predict 1 day ahead', fontsize
plt.legend(fontsize=10)
plt.show()
```



The learning curves show that the validation loss and the training loss are converging to each other with almost no indication of overfitting between them as the number of epochs increase.

(d) [5 marks]

Create a supervised learning dataset suitable for predicting 3 days ahead instead of 1 day ahead. Adjust the deep RNN in (c) so that it predicts 3 days ahead. Use 100 epochs and the Nadam optimizer. Using the test set, report the mean squared error and the accuracy for the movement direction for each of the 3 days ahead predictions. Comment on the result and the learning curve.

[Add your solution here]

```
In [ ]: X_train, y_train = ts_split(train['DEXJPUS'], target_steps=3)
        X_valid, y_valid = ts_split(validation['DEXJPUS'], target_steps=3)
        X_test, y_test = ts_split(test['DEXJPUS'], target_steps=3)
```

```
In [ ]: reset_session()

        model = tf.keras.models.Sequential([SimpleRNN(20, return_sequences=True),
                                             SimpleRNN(20),
```

```

Dense(3)])

model.compile(loss='mse', optimizer='nadam')

history_3 = model.fit(X_train[..., np.newaxis], y_train, epochs=100,
                      validation_data = (X_valid[..., np.newaxis], y_v

y_test_pred = model.predict(X_test[..., np.newaxis])

```

/Users/eunicehan/.pyenv/versions/3.10.11/lib/python3.10/site-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(**kwargs)
```

55/55 ————— 0s 2ms/step

```

In [ ]: for i in range(3):
        mse_rnn = mean_squared_error(y_test[:, i], y_test_pred[:, i])
        print(f"The Mean Squared Error of the RNN model that predicted 3 d
movement_test = X_test[:, -1] < y_test[:, i].ravel()
movement_pred = X_test[:, -1] < y_test_pred[:, i].ravel()
movement_accuracy = accuracy_score(movement_test, movement_pred)
        print(f'The Accuracy of the Movement direction for Day {i+1} ahead
        print("")

```

The Mean Squared Error of the RNN model that predicted 3 days ahead of Day 1: 0.6723747640663801

The Accuracy of the Movement direction for Day 1 ahead prediction: 0.4795389048991354

The Mean Squared Error of the RNN model that predicted 3 days ahead of Day 2: 1.1163307622954122

The Accuracy of the Movement direction for Day 2 ahead prediction: 0.47089337175792506

The Mean Squared Error of the RNN model that predicted 3 days ahead of Day 3: 1.5430068212967971

The Accuracy of the Movement direction for Day 3 ahead prediction: 0.4899135446685879

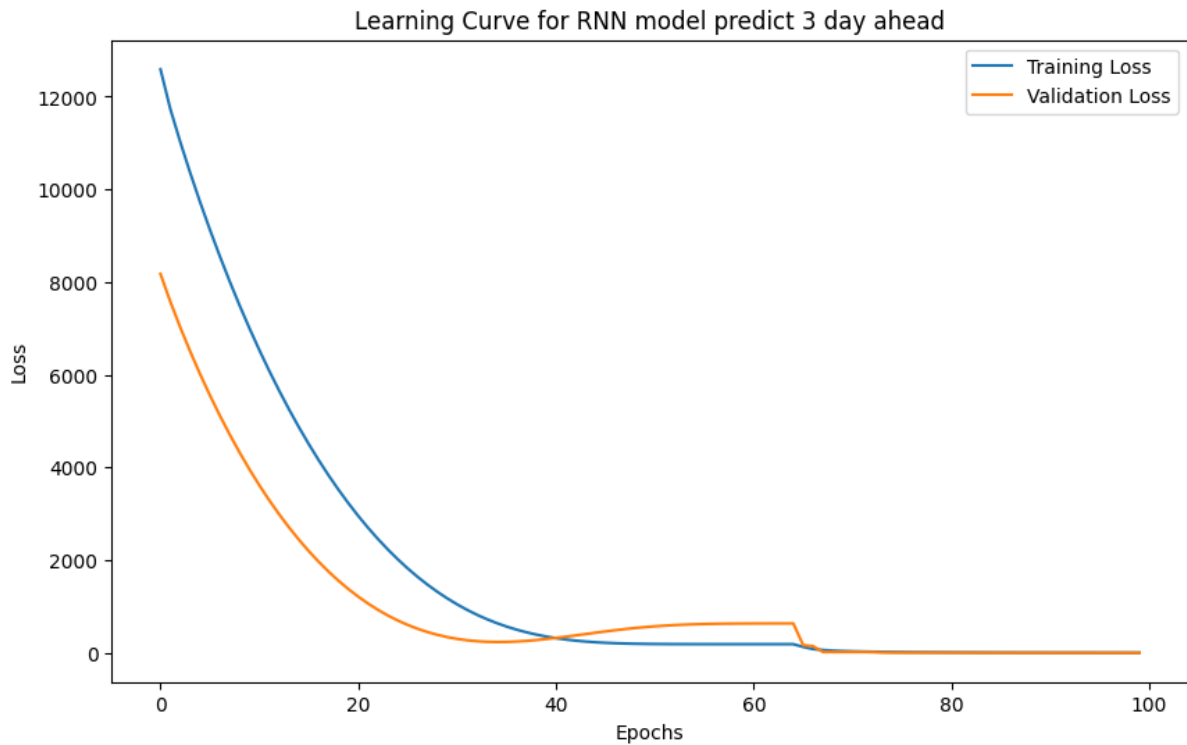
After getting the mean squared error and the accuracy of the movement direction of predicting up to 3 days ahead, I could observe that the mean squared error is increasing as the forecast horizon extends. I can also see that the accuracy of the movement direction tends to fluctuate around chance level. This can tell that it is easier to predict the days closer to the present and it is more difficult to predict further days into the future.

```

In [ ]: plt.figure(figsize=(10,6))
        plt.plot(history_3.history['loss'], label='Training Loss')

```

```
plt.plot(history_3.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Learning Curve for RNN model predict 3 day ahead')
plt.legend()
plt.show()
```



The learning curve for predicting 3 days ahead model is having a similar shape and pattern to predicting 1 day ahead model in part (c). The learning curves of the training loss and validation loss are converging and is showing almost no indication of overfitting as the number of epochs is increasing.